

Estudo 3

Fredeico Pedrosa

2026-07-31

Contents

1	Processamento de dados	2
1.1	Sociodemográficos	2
1.2	BFI	3
1.3	DASS-21	4
1.4	ALMA	4
2	Análises descritivas	5
2.1	Escolaridade vs Treino Musical	8
2.2	Nível de estudo musical por área de trabalho	10
3	Levantamento de validade	12
3.1	DASS-21	12
3.2	BFI-2	15
4	ALMA	43
4.1	Dados	43
4.2	PLS-SEM	44
4.3	PCA	50
4.4	Análise Exploratória de Grafos Hierárquica	51
4.5	CFA	51
4.6	Relações entre os modelos de mensuração	54

```
library(tidyverse)
```

```
## Warning: pacote 'ggplot2' foi compilado no R versão 4.5.2
```

```
## Warning: pacote 'tibble' foi compilado no R versão 4.5.2
```

```
## Warning: pacote 'readr' foi compilado no R versão 4.5.2
```

```
## Warning: pacote 'purrr' foi compilado no R versão 4.5.2

## Warning: pacote 'stringr' foi compilado no R versão 4.5.2

## Warning: pacote 'forcats' foi compilado no R versão 4.5.2

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.6
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.2      v tibble    3.3.1
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.2.1
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readxl)
library(psych)
```

```
##
## Anexando pacote: 'psych'
##
## Os seguintes objetos são mascarados por 'package:ggplot2':
##
##      %>%, alpha
```

```
data <- read_excel("~/PIBIT2025/pesquisa2026a.xlsx")
```

1 Processamento de dados

1.1 Sociodemográficos

```
data_clean <- data %>%
  # 1. Renomeação padronizada
  rename(
    timestamp = 1, email = 2, interest = 3, consent = 4,
    age = 5, sex = 6, gender_raw = 7, education_raw = 8,
    music_raw = 9, professional_field = 10
  ) %>%
  mutate(
    # 2. Idade numérica
    age = as.numeric(age),

    # 3. Gênero com Rigor (Inválidos/Orientação -> NA)
    gender_identity = case_when(
      gender_raw %in% c("Cisgênero", "Feminino", "Mulher") & sex == "Feminino" ~ "Cis_Woman",
      gender_raw %in% c("Cisgênero", "Homem") & sex == "Masculino" ~ "Cis_Man",
      gender_raw == "Não-binário" ~ "Non_Binary",
```

```

    gender_raw == "Transgênero" ~ "Trans_Person",
    TRUE ~ NA_character_
  ),

  # 4. Escolaridade (Fator Ordenado)
  education_level = factor(case_when(
    education_raw %in% c("Ensino médio", "Técnico", "Superior incompleto") ~ "High_School",
    education_raw %in% c("Graduação", "Graduada em Musicoterapia") ~ "Undergraduate",
    education_raw %in% c("Especialização", "Pós graduação", "Pós Graduação", "Pós graduado", "Pós Gra
    education_raw %in% c("Mestrado", "Mestrado em andamento") ~ "Master",
    education_raw %in% c("Doutorado", "Pós-doutorado") ~ "Doctorate",
    TRUE ~ "Other"
  ), levels = c("High_School", "Undergraduate", "Postgrad_Spec", "Master", "Doctorate"), ordered = TRUE),

  # 5. Tempo de Estudo Musical (Com recuperação de dados)
  music_training = factor(case_when(
    professional_field %in% c("20 anos", "hobby") ~ "5yrs_plus",
    music_raw %in% c("Não estudei música", "Noção escolar") ~ "None",
    music_raw %in% c("De 1 mês a 1 ano", "Faço aulas de percussão...") ~ "1mo_to_1yr",
    music_raw == "De 1 ano a 5 anos" ~ "1yr_to_5yrs",
    music_raw == "5 anos ou mais" | grepl("11 anos|23 anos|Muitos anos", music_raw) ~ "5yrs_plus",
    TRUE ~ "None"
  ), levels = c("None", "1mo_to_1yr", "1yr_to_5yrs", "5yrs_plus"), ordered = TRUE),

  # 6. NOVO: Agrupamento de Setores Profissionais (Regex)
  work_sector = case_when(
    grepl("Música|Musica|Musicista|Músico|musico|Musicoterapeuta|Musicoterapia|Artes|Dança|Arte", professional_field) ~ "Music",
    grepl("Saúde|Saude|Psicolog|Médico|Enfermagem|Farmacêutica|Otorrino|Veterinário|Terapeuta", professional_field) ~ "Health",
    grepl("Educação|Educador|Professor|Pedagogia|Docência|Letras", professional_field, ignore.case = TRUE) ~ "Education",
    grepl("Engenharia|Tecnologia|TI|Sistemas|Computação|Industrial|Mecânica|Soldador|Metalurgia|Mineração", professional_field) ~ "Engineering",
    grepl("Administra|Advogado|Jurídica|Comercial|Vendedor|Contabilidade|Financeiro|Logística|Serviço", professional_field) ~ "Business",
    grepl("Estudante|Universitária|Aposentada|Desempregado|Do lar|Dona de casa", professional_field, ignore.case = TRUE) ~ "Others"
  ),
  work_sector = as.factor(work_sector)
)

```

1.2 BFI

```

# Renomear colunas 11 a 70
bfi_names <- paste0("BFI", sprintf("%02d", 1:60))
colnames(data_clean)[11:70] <- bfi_names

likert_bfi <- c("Discordo totalmente"=1, "Discordo em parte"=2, "Nem concordo nem discordo"=3, "Concordo totalmente"=4)
reversed_bfi <- c("BFI11", "BFI16", "BFI26", "BFI31", "BFI36", "BFI51", "BFI12", "BFI17", "BFI22", "BFI27", "BFI32", "BFI37", "BFI41", "BFI46", "BFI56", "BFI61", "BFI66", "BFI71")

data_clean <- data_clean %>%
  mutate(across(starts_with("BFI"), ~ as.numeric(likert_bfi[.x]))) %>%
  mutate(across(all_of(reversed_bfi), ~ 6 - .x)) %>%
  mutate( # Cálculo de Médias dos Fatores
    bfi_E = rowMeans(select(., BFI01, BFI06, BFI11, BFI16, BFI21, BFI26, BFI31, BFI36, BFI41, BFI46, BFI51, BFI56, BFI61, BFI66, BFI71)),
    bfi_A = rowMeans(select(., BFI02, BFI07, BFI12, BFI17, BFI22, BFI27, BFI32, BFI37, BFI42, BFI47, BFI52, BFI57, BFI62, BFI67, BFI72))
  )

```

```

bfi_C = rowMeans(select(., BFI03, BFI08, BFI13, BFI18, BFI23, BFI28, BFI33, BFI38, BFI43, BFI48, BF
bfi_N = rowMeans(select(., BFI04, BFI09, BFI14, BFI19, BFI24, BFI29, BFI34, BFI39, BFI44, BFI49, BF
bfi_O = rowMeans(select(., BFI05, BFI10, BFI15, BFI20, BFI25, BFI30, BFI35, BFI40, BFI45, BFI50, BF
)

```

1.3 DASS-21

```

# 1. Renomeação para DASS01 a DASS21
dass_names <- paste0("DASS", sprintf("%02d", 1:21))
colnames(data_clean)[92:112] <- dass_names

# 1. Mapa de Conversão 1-4 (para Análise Fatorial costuma ser melhor que 0-3)
# Se preferir 0-3, basta trocar os números abaixo
likert_dass_fatorial <- c(
  "Nunca" = 1,
  "Às vezes" = 2,
  "Frequentemente" = 3,
  "Sempre" = 4
)

# 2. Criando as colunas DASS direto do dado bruto (original 'data')
# Isso evita o erro de herdar NAs do data_clean anterior
for(i in 1:21) {
  orig_col <- 91 + i # As colunas DASS começam na 92
  new_name <- paste0("DASS", sprintf("%02d", i))
  data_clean[[new_name]] <- as.numeric(likert_dass_fatorial[data[[orig_col]]])
}

```

1.4 ALMA

```

# 1. Definição dos nomes das subvariáveis (11 por trecho)
sub_names <- c("ativo", "energ", "atent", "relax", "sonol",
               "prazer", "felic", "humor", "desag", "trist", "txt")

# 2. Gerando todos os nomes de T01 a T10
all_musical_names <- map(1:10, ~ paste0("T", sprintf("%02d", .x), "_", sub_names)) %>% unlist()

# 3. Aplicando a renomeação ao data_clean
colnames(data_clean)[113:222] <- all_musical_names

# 4. Mapa de Conversão de Intensidade (1 a 5)
likert_intensidade <- c(
  "Nem um pouco" = 1,
  "Um pouco" = 2,
  "Moderadamente" = 3,
  "Muito" = 4,
  "Extremamente" = 5
)

```

```

# 5. Conversão Numérica (Exceto as colunas de texto '_txt')
numeric_cols <- all_musical_names[!str_detect(all_musical_names, "_txt")]

# Buscando do dado original 'data' para evitar herança de NAs
for(col_name in all_musical_names) {
  orig_col_idx <- which(colnames(data_clean) == col_name)

  if(str_detect(col_name, "_txt")) {
    # Mantém como texto
    data_clean[[col_name]] <- data[[orig_col_idx]]
  } else {
    # Converte para numérico usando o mapa
    data_clean[[col_name]] <- as.numeric(likert_intensidade[data[[orig_col_idx]])]
  }
}

### INVERSÃO

# 1. Definindo os sufixos que devem ser invertidos
sufixos_reversos <- c("_relax", "_sonol", "_desag", "_trist")

# 2. Identificando todas as colunas que terminam com esses sufixos
cols_para_inverter <- names(data_clean) %>%
  keep(~ any(str_ends(.x, sufixos_reversos)))

# 3. Aplicando a inversão (6 - x)
data_clean <- data_clean %>%
  mutate(across(all_of(cols_para_inverter), ~ 6 - .x))

```

2 Análises descritivas

```

library(tidyverse)
library(psych)

# --- A. Tabela de Frequências para Variáveis Categóricas ---
# Sexo, Identidade de Gênero, Escolaridade e Treino Musical
desc_categoricas <- data_clean %>%
  dplyr::select(sex, gender_identity, education_level, music_training, work_sector) %>%
  gather(variable, value) %>%
  count(variable, value) %>%
  group_by(variable) %>%
  mutate(percentage = (n / sum(n)) * 100)

```

```

## Warning: attributes are not identical across measure variables; they will be
## dropped

```

```

print(desc_categoricas)

```

```

## # A tibble: 24 x 4

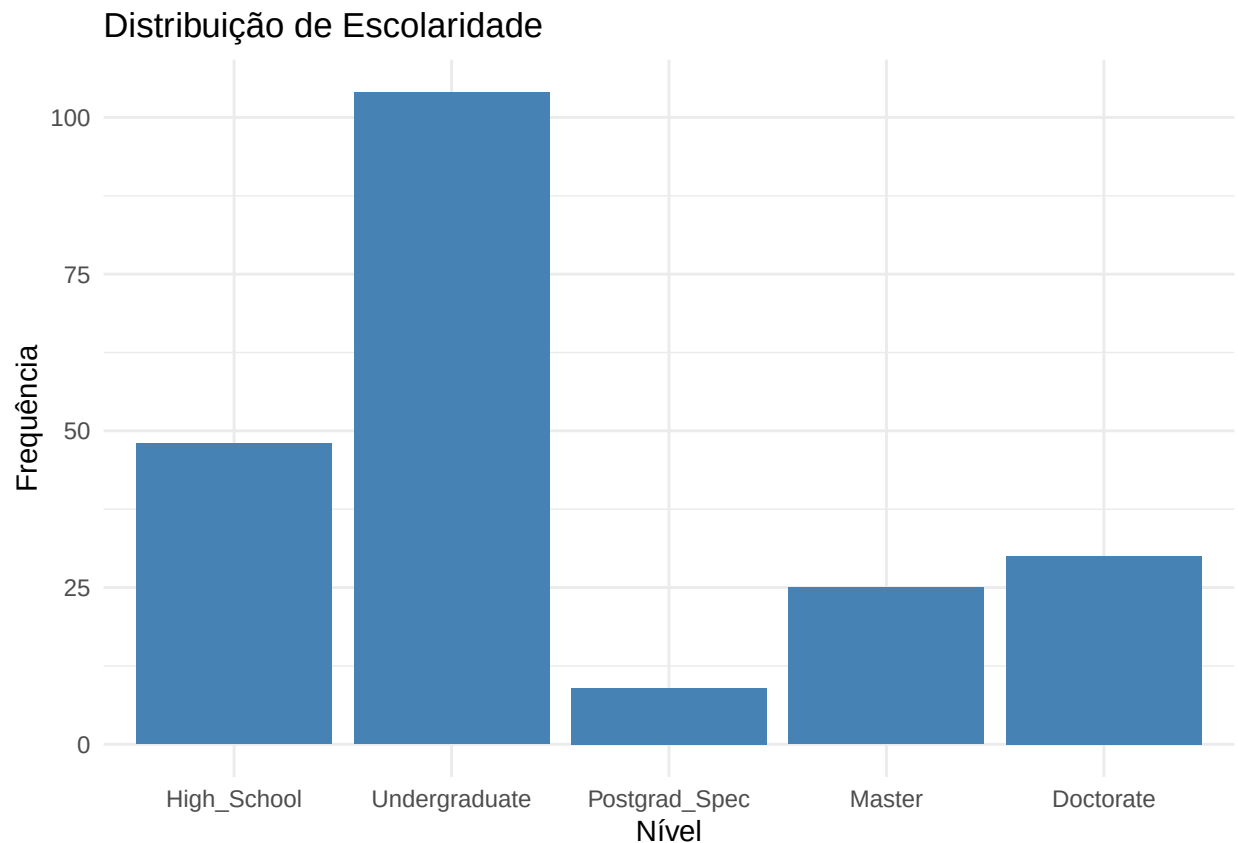
```

```
## # Groups:   variable [5]
##   variable      value      n percentage
##   <chr>        <chr>    <int>    <dbl>
## 1 education_level Doctorate      30     13.9
## 2 education_level High_School    48     22.2
## 3 education_level Master         25     11.6
## 4 education_level Postgrad_Spec   9      4.17
## 5 education_level Undergraduate 104     48.1
## 6 gender_identity Cis_Man        82     38.0
## 7 gender_identity Cis_Woman     116     53.7
## 8 gender_identity Non_Binary      2      0.926
## 9 gender_identity Trans_Person    1      0.463
## 10 gender_identity <NA>         15      6.94
## # i 14 more rows
```

```
# --- B. Estatística Descritiva para Idade ---
describe(data_clean$age)
```

```
##   vars    n mean    sd median trimmed  mad min max range skew kurtosis se
## X1      1 216 39.58 14.66     38  38.75 16.31 17  75    58 0.41   -0.78  1
```

```
# --- C. Visualização Estratégica (Opcional, mas ajuda no relatório) ---
ggplot(data_clean, aes(x = education_level)) +
  geom_bar(fill = "steelblue") +
  theme_minimal() +
  labs(title = "Distribuição de Escolaridade", x = "Nível", y = "Frequência")
```



Mais mulheres ($p < 0,05$)

```
# Teste de Qui-Quadrado para aderência (Equilíbrio de Gênero)
sex_table <- table(data_clean$sex)
# Removendo "Prefiro não declarar" para o teste
sex_table <- sex_table[names(sex_table) %in% c("Feminino", "Masculino")]
chisq.test(sex_table)
```

```
##
## Chi-squared test for given probabilities
##
## data: sex_table
## X-squared = 5.6977, df = 1, p-value = 0.01699
```

```
library(psych)
```

```
# A. Descritiva Geral das Categóricas
data_clean %>%
  select(sex, gender_identity, education_level, music_training, work_sector) %>%
  summary()
```

```
##      sex      gender_identity      education_level      music_training
## Length:216      Length:216      High_School : 48      None :91
## Class :character      Class :character      Undergraduate:104      1mo_to_1yr :22
## Mode :character      Mode :character      Postgrad_Spec: 9      1yr_to_5yrs:23
##                                     Master : 25      5yrs_plus :80
##                                     Doctorate : 30
##
##
##
##      work_sector
## Admin_Law_Business:11
## Education :39
## Health :51
## Music_Arts :40
## Others :39
## STEM_Industry :27
## Student_Unemployed: 9
```

```
# B. Verificação de Gênero (Reporte os NAs no seu paper)
table(data_clean$gender_identity, useNA = "always")
```

```
##
##      Cis_Man      Cis_Woman      Non_Binary      Trans_Person      <NA>
##      82      116      2      1      15
```

```
# C. Teste de Diferença de Idade entre Sexos (Normalidade check)
# Verificando se a amostra é equilibrada em idade
t.test(age ~ sex, data = data_clean %>% filter(sex %in% c("Feminino", "Masculino")))
```

```
##
## Welch Two Sample t-test
```

```
##
## data: age by sex
## t = 0.62902, df = 201.48, p-value = 0.53
## alternative hypothesis: true difference in means between group Feminino and group Masculino is not e
## 95 percent confidence interval:
## -2.680274 5.191386
## sample estimates:
## mean in group Feminino mean in group Masculino
## 40.20000 38.94444
```

```
# D. Cruzamento: Escolaridade vs. Treino Musical (Estratégico!)
table(data_clean$education_level, data_clean$music_training)
```

```
##
##           None 1mo_to_1yr 1yr_to_5yrs 5yrs_plus
## High_School   32         2         4        10
## Undergraduate 43        12        13        36
## Postgrad_Spec  5         1         1         2
## Master         6         6         3        10
## Doctorate      5         1         2        22
```

2.1 Escolaridade vs Treino Musical

```
library(tidyverse)

data_clean <- data_clean %>%
  mutate(education_grouped = fct_collapse(education_level,
    Undergrad_Spec = c("Undergraduate", "Postgrad_Spec"),
    High_School = "High_School",
    Master = "Master",
    Doctorate = "Doctorate"
  ))

teste_assoc <- chisq.test(data_clean$education_level, data_clean$music_training, simulate.p.value = TRUE)
print(teste_assoc)
```

```
##
## Pearson's Chi-squared test with simulated p-value (based on 2000
## replicates)
##
## data: data_clean$education_level and data_clean$music_training
## X-squared = 37.695, df = NA, p-value = 0.001499
```

```
# A força da associação
```

```
library(lsr)
```

```
## Warning: pacote 'lsr' foi compilado no R versão 4.5.2
```



```
cramersV(data_clean$education_level, data_clean$music_training)
```

```
## Warning in stats::chisq.test(...): Aproximação do qui-quadrado pode estar  
## incorreta
```

```
## [1] 0.2411875
```

```
# 1. Gerar a tabela com os novos grupos
```

```
tab_final <- table(data_clean$education_grouped, data_clean$music_training)
```

```
# 2. Rodar o Qui-Quadrado
```

```
teste_final <- chisq.test(tab_final, simulate.p.value = TRUE)
```

```
# 3. Extrair os Resíduos Padronizados (O Post Hoc real)
```

```
residuos <- teste_final$residuals
```

```
print(round(residuos, 2))
```

```
##
```

```
##           None 1mo_to_1yr 1yr_to_5yrs 5yrs_plus  
## High_School  2.62    -1.31    -0.49    -1.84  
## Undergrad_Spec 0.06     0.44     0.57    -0.60  
## Master      -1.40     2.16     0.21     0.24  
## Doctorate   -2.15    -1.18    -0.67     3.27
```

```
# 4. Visualização de Calor (Para o seu relatório)
```

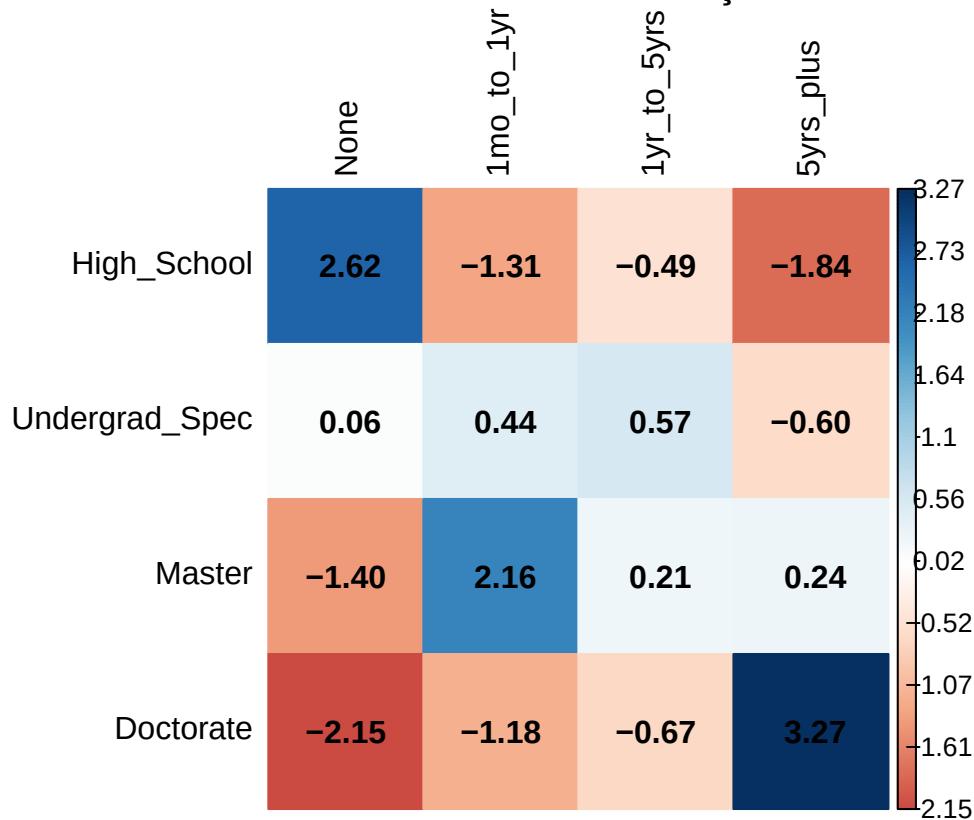
```
# install.packages("corrplot")
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
corrplot(residuos, is.cor = FALSE, method = "color",  
          tl.col = "black", addCoef.col = "black",  
          title = "Post Hoc: Onde está a associação?",  
          mar=c(0,0,1,0))
```

Post Hoc: Onde está a associação?



2.2 Nível de estudo musical por área de trabalho

```
library(tidyverse)
library(lsr)
library(corrplot)

# 1. Gerar a tabela cruzada
tab_work_music <- table(data_clean$work_sector, data_clean$music_training)

# 2. Rodar o Qui-Quadrado (com simulação devido a células pequenas)
teste_work <- chisq.test(tab_work_music, simulate.p.value = TRUE)
print(teste_work)

##
## Pearson's Chi-squared test with simulated p-value (based on 2000
## replicates)
##
## data:  tab_work_music
## X-squared = 71.873, df = NA, p-value = 0.0004998

# 3. Medir a força da associação (Cramer's V)
cv_work <- cramersV(tab_work_music)
```

```
## Warning in stats::chisq.test(...): Aproximação do qui-quadrado pode estar
## incorreta
```

```
cat("Cramer's V:", round(cv_work, 3), "\n")
```

```
## Cramer's V: 0.333
```

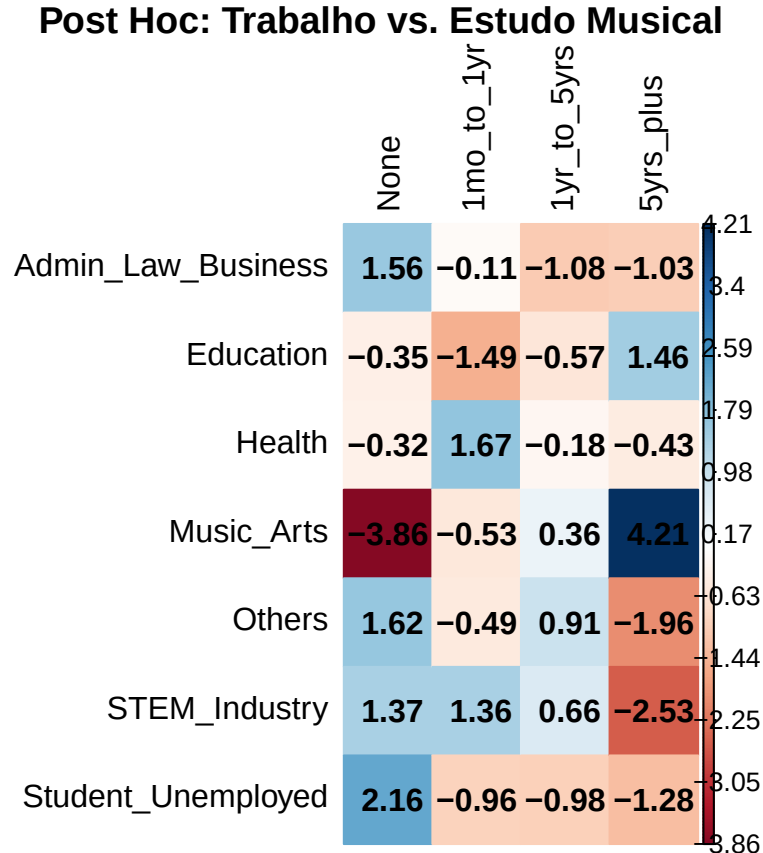
```
# 4. Extrair os Resíduos Padronizados (Post Hoc)
```

```
residuos_work <- teste_work$residuals
print(round(residuos_work, 2))
```

```
##
##           None 1mo_to_1yr 1yr_to_5yrs 5yrs_plus
## Admin_Law_Business 1.56 -0.11 -1.08 -1.03
## Education -0.35 -1.49 -0.57 1.46
## Health -0.32 1.67 -0.18 -0.43
## Music_Arts -3.86 -0.53 0.36 4.21
## Others 1.62 -0.49 0.91 -1.96
## STEM_Industry 1.37 1.36 0.66 -2.53
## Student_Unemployed 2.16 -0.96 -0.98 -1.28
```

```
# 5. Visualização de Calor
```

```
corrplot(residuos_work, is.cor = FALSE, method = "color",
          tl.col = "black", addCoef.col = "black",
          title = "Post Hoc: Trabalho vs. Estudo Musical",
          mar=c(0,0,1,0))
```



3 Levantamento de validade

3.1 DASS-21

```
library(lavaan)
```

```
## Warning: pacote 'lavaan' foi compilado no R versão 4.5.1
```

```
## This is lavaan 0.6-20  
## lavaan is FREE software! Please report any bugs.
```

```
##  
## Anexando pacote: 'lavaan'
```

```
## O seguinte objeto é mascarado por 'package:psych':  
##  
##      cor2cov
```

```
library(semPlot)
```

```
## Warning: pacote 'semPlot' foi compilado no R versão 4.5.2
```

```
# Definindo as variáveis para facilitar a escrita do modelo  
dass_items <- paste0("DASS", sprintf("%02d", 1:21))
```

```
# --- MODELO 1: TRIFATORIAL CORRELACIONADO (CLÁSSICO) ---
```

```
mod_trifatorial <- '  
  ANX =~ DASS02 + DASS04 + DASS07 + DASS09 + DASS15 + DASS19 + DASS20  
  STR =~ DASS01 + DASS06 + DASS08 + DASS11 + DASS12 + DASS14 + DASS18  
  DEP =~ DASS03 + DASS05 + DASS10 + DASS13 + DASS16 + DASS17 + DASS21  
,
```

```
fit_trifatorial <- cfa(mod_trifatorial, data = data_clean, ordered = dass_items,  
  estimator = "WLSMV", std.lv = TRUE)
```

```
# --- MODELO 2: BIFATOR (A BUSCA PELO P-FACTOR / FG) ---
```

```
# Aqui fixamos a ortogonalidade para isolar a variância do Fator Geral
```

```
mod_bifactor <- '  
  FG =~ DASS01 + DASS02 + DASS03 + DASS04 + DASS05 + DASS06 + DASS07 +  
        DASS08 + DASS09 + DASS10 + DASS11 + DASS12 + DASS13 + DASS14 +  
        DASS15 + DASS16 + DASS17 + DASS18 + DASS19 + DASS20 + DASS21  
  
  ANX =~ DASS02 + DASS04 + DASS07 + DASS09 + DASS15 + DASS19 + DASS20  
  STR =~ DASS01 + DASS06 + DASS08 + DASS11 + DASS12 + DASS14 + DASS18  
  DEP =~ DASS03 + DASS05 + DASS10 + DASS13 + DASS16 + DASS17 + DASS21  
,
```

```
fit_bifactor <- cfa(mod_bifactor, data = data_clean, ordered = dass_items,  
  estimator = "WLSMV", std.lv = TRUE, orthogonal = TRUE)
```

```
# --- MODELO 3: UNIDIMENSIONAL ---
mod_unidim <- '
  FG =~ DASS01 + DASS02 + DASS03 + DASS04 + DASS05 + DASS06 + DASS07 +
        DASS08 + DASS09 + DASS10 + DASS11 + DASS12 + DASS13 + DASS14 +
        DASS15 + DASS16 + DASS17 + DASS18 + DASS19 + DASS20 + DASS21
'

fit_unidim <- cfa(mod_unidim, data = data_clean, ordered = dass_items,
  estimator = "WLSMV", std.lv = TRUE)
```

3.1.1 Índices de ajuste

```
# Função para extrair medidas de ajuste
get_fits <- function(fit_obj, model_name) {
  measures <- fitMeasures(fit_obj, c("chisq", "df", "cfi", "tli", "srmr", "rmsea",
    "rmsea.ci.lower", "rmsea.ci.upper"))
  return(c(Model = model_name, round(measures, 3)))
}

# Comparação de Modelos
tabela_comparativa <- rbind(
  get_fits(fit_trifatorial, "Trifatorial"),
  get_fits(fit_bifactor, "Bifactor (p-factor)"),
  get_fits(fit_unidim, "Unidimensional")
)

print(tabela_comparativa)
```

```
##      Model                chisq    df    cfi    tli    srmr    rmsea
## [1,] "Trifatorial"          "216.621" "186" "0.998" "0.998" "0.059" "0.028"
## [2,] "Bifactor (p-factor)"  "144.773" "168" "1"      "1.002" "0.05"  "0"
## [3,] "Unidimensional"      "538.222" "189" "0.977" "0.974" "0.092" "0.093"
##      rmsea.ci.lower rmsea.ci.upper
## [1,] "0"            "0.042"
## [2,] "0"            "0.014"
## [3,] "0.084"        "0.102"
```

3.1.2 Fidedignidade Composta

```
library(semTools)

##

## #####

## This is semTools 0.5-7

## All users of R (or SEM) are invited to submit functions or ideas for functions.
```

```
## #####
```

```
##
```

```
## Anexando pacote: 'semTools'
```

```
## Os seguintes objetos são mascarados por 'package:psych':
```

```
##
```

```
##     reliability, skew
```

```
## O seguinte objeto é mascarado por 'package:readr':
```

```
##
```

```
##     clipboard
```

```
compRelSEM(fit_trifatorial)
```

```
##     ANX     STR     DEP
```

```
## 0.888 0.867 0.889
```

```
compRelSEM(fit_bifactor)
```

```
##     FG     ANX     STR     DEP
```

```
## 0.850 0.142 0.177 0.302
```

```
compRelSEM(fit_unidim)
```

```
##     FG
```

```
## 0.975
```

3.1.3 Extração dos escores de p-factor

```
# Extraindo escores fatoriais do modelo Bifactor
```

```
escores_dass <- lavPredict(fit_bifactor)
```

```
# Adicionando o p-factor (FG) ao banco principal
```

```
data_clean$dass_p_factor <- escores_dass[, "FG"]
```

```
# Verificação: correlação do p-factor com a idade (lembra que deu sig. antes?)
```

```
cor.test(data_clean$dass_p_factor, data_clean$age, method = "pearson")
```

```
##
```

```
## Pearson's product-moment correlation
```

```
##
```

```
## data: data_clean$dass_p_factor and data_clean$age
```

```
## t = -4.1227, df = 214, p-value = 5.357e-05
```

```
## alternative hypothesis: true correlation is not equal to 0
```

```
## 95 percent confidence interval:
```

```
## -0.3906031 -0.1429369
```

```
## sample estimates:
```

```
##     cor
```

```
## -0.2712541
```

3.2 BFI-2

```
library(lavaan)

# Definindo os itens por fator
E_items <- paste0("BFI", sprintf("%02d", c(1, 6, 11, 16, 21, 26, 31, 36, 41, 46, 51, 56)))
A_items <- paste0("BFI", sprintf("%02d", c(2, 7, 12, 17, 22, 27, 32, 37, 42, 47, 52, 57)))
C_items <- paste0("BFI", sprintf("%02d", c(3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58)))
N_items <- paste0("BFI", sprintf("%02d", c(4, 9, 14, 19, 24, 29, 34, 39, 44, 49, 54, 59)))
O_items <- paste0("BFI", sprintf("%02d", c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60)))

all_bfi_items <- c(E_items, A_items, C_items, N_items, O_items)

# --- MODELO 1: 5 FATORES CORRELACIONADOS ---
mod_bfi_5 <- paste0('
  Extraversion      =~ ', paste(E_items, collapse = " + "), '
  Agreeableness     =~ ', paste(A_items, collapse = " + "), '
  Conscientiousness =~ ', paste(C_items, collapse = " + "), '
  Negative_Emot     =~ ', paste(N_items, collapse = " + "), '
  Open_Mindedness   =~ ', paste(O_items, collapse = " + ")
)

fit_bfi_5 <- cfa(mod_bfi_5, data = data_clean, ordered = all_bfi_items,
  estimator = "WLSMV", std.lv = TRUE)
```

```
## Warning: lavaan->lav_model_vcov():
##   The variance-covariance matrix of the estimated parameters (vcov) does not
##   appear to be positive definite! The smallest eigenvalue (= -6.930907e-17)
##   is smaller than zero. This may be a symptom that the model is not
##   identified.
```

```
# --- MODELO 2: BIFACTOR (Fator Geral de Personalidade - GFP) ---
mod_bfi_gfp <- paste0('
  GFP =~ ', paste(all_bfi_items, collapse = " + "), '
  Extraversion      =~ ', paste(E_items, collapse = " + "), '
  Agreeableness     =~ ', paste(A_items, collapse = " + "), '
  Conscientiousness =~ ', paste(C_items, collapse = " + "), '
  Negative_Emot     =~ ', paste(N_items, collapse = " + "), '
  Open_Mindedness   =~ ', paste(O_items, collapse = " + ")
)

fit_bfi_gfp <- cfa(mod_bfi_gfp, data = data_clean, ordered = all_bfi_items,
  estimator = "WLSMV", std.lv = TRUE, orthogonal = TRUE)
```

```
## Warning: lavaan->lav_model_vcov():
##   The variance-covariance matrix of the estimated parameters (vcov) does not
##   appear to be positive definite! The smallest eigenvalue (= -6.831583e-17)
##   is smaller than zero. This may be a symptom that the model is not
##   identified.
```

```

mod_bfi_hier <- paste0('
  Extraversion      =~ ', paste(E_items, collapse = " + "), '
  Agreeableness     =~ ', paste(A_items, collapse = " + "), '
  Conscientiousness =~ ', paste(C_items, collapse = " + "), '
  Negative_Emot      =~ ', paste(N_items, collapse = " + "), '
  Open_Mindedness   =~ ', paste(O_items, collapse = " + "), '

  GFP =~ Extraversion + Agreeableness + Conscientiousness + Negative_Emot + Open_Mindedness'
)

fit_bfi_hier <- cfa(mod_bfi_hier, data = data_clean, ordered = all_bfi_items,
  estimator = "WLSMV", std.lv = TRUE, orthogonal = TRUE)

```

```

## Warning: lavaan->lav_model_vcov():
##   The variance-covariance matrix of the estimated parameters (vcov) does not
##   appear to be positive definite! The smallest eigenvalue (= -3.970201e-15)
##   is smaller than zero. This may be a symptom that the model is not
##   identified.

```

3.2.1 Índices de ajuste

```

# Função de extração de fit (incluindo seus favoritos)
get_bfi_fits <- function(fit_obj, model_name) {
  if (lavInspect(fit_obj, "converged")) {
    measures <- fitMeasures(fit_obj, c("chisq", "df", "cfi", "tli", "srmr", "rmsea", "rmsea.ci.lower", "rmsea.ci.upper"))
    return(c(Model = model_name, round(measures, 3)))
  } else {
    return(c(Model = model_name, "DNC")) # Did Not Converge
  }
}

tabela_bfi <- rbind(
  get_bfi_fits(fit_bfi_5, "BFI 5-Factors"),
  get_bfi_fits(fit_bfi_gfp, "BFI GFP (Bifactor)"),
  get_bfi_fits(fit_bfi_hier, "BFI Hierarchical (2orf)")
)

print(tabela_bfi)

```

```

##      Model                chisq      df      cfi      tli      srmr
## [1,] "BFI 5-Factors"      "7166.916" "1700" "0.761" "0.751" "0.136"
## [2,] "BFI GFP (Bifactor)"  "4977.006" "1650" "0.855" "0.844" "0.115"
## [3,] "BFI Hierarchical (2orf)" "7231.001" "1705" "0.758" "0.749" "0.137"
##      rmsea  rmsea.ci.lower rmsea.ci.upper
## [1,] "0.122" "0.119"      "0.125"
## [2,] "0.097" "0.094"      "0.1"
## [3,] "0.123" "0.12"       "0.126"

```

3.2.2 TRI Rasch TRI para cada fator do Big Five


```
library(eRm)
```

3.2.2.1 Extroversão

```
## Warning: pacote 'eRm' foi compilado no R versão 4.5.1
```

```
##
```

```
## Anexando pacote: 'eRm'
```

```
## O seguinte objeto é mascarado por 'package:psych':
```

```
##
```

```
##      sim.rasch
```

```
library(dplyr)
```

```
# Definindo os vetores de itens (baseado na estrutura 1-60 do BFI-2)
```

```
E_items <- paste0("BFI", sprintf("%02d", c(1, 6, 11, 16, 21, 26, 31, 36, 41, 46, 51, 56)))
```

```
A_items <- paste0("BFI", sprintf("%02d", c(2, 7, 12, 17, 22, 27, 32, 37, 42, 47, 52, 57)))
```

```
C_items <- paste0("BFI", sprintf("%02d", c(3, 8, 13, 18, 23, 28, 33, 38, 43, 48, 53, 58)))
```

```
N_items <- paste0("BFI", sprintf("%02d", c(4, 9, 14, 19, 24, 29, 34, 39, 44, 49, 54, 59)))
```

```
O_items <- paste0("BFI", sprintf("%02d", c(5, 10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60)))
```

```
data_E <- na.omit(data_clean[, E_items])
```

```
pcm_E <- PCM(data_E)
```

```
## Warning:
```

```
## The following items have no 0-responses:
```

```
## BFI01 BFI06 BFI11 BFI16 BFI21 BFI26 BFI31 BFI36 BFI41 BFI46 BFI51 BFI56
```

```
## Responses are shifted such that lowest category is 0.
```

```
thres_E <- thresholds(pcm_E)
```

```
print(thres_E$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3 Threshold 4
## BFI01 -0.57378518 -2.5094197  0.40745696 -0.94414487  0.7509669
## BFI06 -0.11558617 -1.6176183 -0.14439484 -0.05345244  1.3531209
## BFI11  0.12077386 -1.1146223  0.17391092  0.29886603  1.1249408
## BFI16  0.93745071  0.1050779  1.18061087  1.06380687  1.4003073
## BFI21  0.27941760 -0.6178838  0.15562516  0.25819628  1.3217328
## BFI26  0.24091484 -0.3189639  0.23737005  0.28306778  0.7621855
## BFI31  0.95536123  0.2435089  1.42125507  0.80438590  1.3522951
## BFI36  0.63581739 -0.5174703  0.45838953  0.74818438  1.8541660
## BFI41  0.26132880 -0.6522460  0.46220635  0.13568538  1.0996694
## BFI46  0.47247969  0.2024251  0.31782383  0.90625145  0.4634183
## BFI51  0.08535586 -1.0174564  0.33191716  0.31555425  0.7114085
## BFI56  0.06060422 -1.4199103  0.04081301  0.36083671  1.2606774
```

```
data_E_revisao <- data_clean %>%
  select(all_of(E_items)) %>%
  mutate(across(everything(), ~ case_when(
    . == 1 ~ 1,          # Discordo totalmente
    . %in% c(2, 3) ~ 2, # Discordo em parte + Neutro
    . == 4 ~ 3,          # Concordo em parte
    . == 5 ~ 4,          # Concordo totalmente
    TRUE ~ NA_real_
  ))) %>%
  as.data.frame() %>%
  na.omit()
```

```
# 3. Rodando o Modelo de Créditos Parciais (PCM)
pcm_E_rev <- PCM(data_E_revisao)
```

```
## Warning:
## The following items have no 0-responses:
## BFI01 BFI06 BFI11 BFI16 BFI21 BFI26 BFI31 BFI36 BFI41 BFI46 BFI51 BFI56
## Responses are shifted such that lowest category is 0.
```

```
# 4. Verificando os Thresholds (O seu objetivo agora)
thres_E_rev <- thresholds(pcm_E_rev)
print(thres_E_rev$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3
## BFI01 -0.67906422 -3.03273120 -0.09461695  1.0901555
## BFI06 -0.04534392 -2.46130657  0.59374282  1.7315320
## BFI11  0.26411899 -1.77377897  1.05986134  1.5062746
## BFI16  1.33450936 -0.09778946  2.29769835  1.8036192
## BFI21  0.47769309 -1.28986949  1.01410163  1.7088471
## BFI26  0.41656292 -0.94963194  1.06359571  1.1357250
## BFI31  1.35383797  0.12261047  2.18667015  1.7522333
## BFI36  0.95386770 -1.02991094  1.63027158  2.2612424
## BFI41  0.44420493 -1.16112024  1.01553288  1.4782021
## BFI46  0.72582324 -0.39208760  1.72085977  0.8486975
## BFI51  0.20766337 -1.59345269  1.13513839  1.0813044
## BFI56  0.18914556 -2.15448633  1.07519733  1.6467257
```

```
data_E_revisao <- data_E_revisao[c(1,2,3,5,6,8,9,12)]
```

```
# 5. Verificando o Item Fit (Para ver quem "ajusta" ao traço)
pcm_E_rev <- PCM(data_E_revisao)
```

```
## Warning:
## The following items have no 0-responses:
## BFI01 BFI06 BFI11 BFI21 BFI26 BFI36 BFI41 BFI56
## Responses are shifted such that lowest category is 0.
```

```
pp_E_rev <- person.parameter(pcm_E_rev)
fit_E_rev <- itemfit(pp_E_rev)
print(fit_E_rev)
```

```
##
## Itemfit Statistics:
##      Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI01 175.673 213  0.971    0.821   0.836  -2.024  -2.018   0.514
## BFI06 195.307 213  0.802    0.913   0.891  -1.029  -1.351   0.438
## BFI11 336.874 213  0.000    1.574   1.486   5.625   5.216  -0.118
## BFI21 154.952 213  0.999    0.724   0.722  -3.494  -3.659   0.638
## BFI26 185.456 213  0.914    0.867   0.886  -1.493  -1.402   0.451
## BFI36 205.503 213  0.631    0.960   0.963  -0.402  -0.393   0.321
## BFI41 162.003 213  0.996    0.757   0.758  -2.998  -3.158   0.612
## BFI56 163.944 213  0.995    0.766   0.781  -2.885  -2.889   0.585
```

```
data_E_revisao <- data_E_revisao[-3]
pcm_E_rev <- PCM(data_E_revisao)
```

```
## Warning:
## The following items have no 0-responses:
## BFI01 BFI06 BFI21 BFI26 BFI36 BFI41 BFI56
## Responses are shifted such that lowest category is 0.
```

```
pp_E_rev <- person.parameter(pcm_E_rev)
fit_E_rev <- itemfit(pp_E_rev)
print(fit_E_rev)
```

```
##
## Itemfit Statistics:
##      Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI01 173.458 211  0.972    0.818   0.847  -1.935  -1.800   0.539
## BFI06 215.159 211  0.408    1.015   0.951   0.195  -0.556   0.437
## BFI21 164.337 211  0.992    0.775   0.768  -2.654  -2.879   0.619
## BFI26 209.170 211  0.523    0.987   1.008  -0.104   0.120   0.422
## BFI36 224.254 211  0.253    1.058   1.067   0.628   0.741   0.298
## BFI41 161.436 211  0.995    0.761   0.761  -2.778  -2.988   0.624
## BFI56 183.538 211  0.914    0.866   0.883  -1.474  -1.404   0.527
```

```
data_A <- na.omit(data_clean[, A_items])
pcm_A <- PCM(data_A)
```

3.2.2.2 Amabilidade

```
## Warning:
## The following items have no 0-responses:
## BFI02 BFI07 BFI12 BFI17 BFI22 BFI27 BFI32 BFI37 BFI42 BFI47 BFI52 BFI57
## Responses are shifted such that lowest category is 0.
```

```
thres_A <- thresholds(pcm_A)
print(thres_A$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3 Threshold 4
## BFI02  0.21910486 -0.82122115  0.4921838 -0.48777018  1.69322696
## BFI07 -0.82380703 -1.12134737 -0.8022089 -0.58776703 -0.78390487
## BFI12  0.53095497 -0.68567392  0.7560372  1.12019093  0.93326563
## BFI17  0.09955386  0.60077780  0.1640306 -0.39074878  0.02415581
## BFI22  0.38149523 -0.77131511  0.8221055  0.61723262  0.85795788
## BFI27  0.26571029 -0.02062981 -0.2496677  0.23935848  1.09378022
## BFI32 -0.58729010 -1.61578901 -0.8319190 -0.59137718  0.68992483
## BFI37  0.51037483 -0.80835437  0.7343857  0.98416136  1.13130664
## BFI42  1.16843761  0.06136024  1.1495399  1.58131558  1.88153474
## BFI47  0.61656508 -0.27410025  0.7001141  0.84810364  1.19214279
## BFI52 -0.48143625 -0.36784010 -0.9919467 -0.08452199      NA
## BFI57  0.42331479 -0.66547863  0.2450321  0.62762664  1.48607904
```

2. Recodificando Amabilidade: Unificando categorias 2 e 3

```
data_A_revisao <- data_clean %>%
  select(all_of(A_items)) %>%
  mutate(across(everything(), ~ case_when(
    . == 1 ~ 1,          # Discordo totalmente
    . %in% c(2, 3) ~ 2, # Discordo em parte + Neutro
    . == 4 ~ 3,          # Concordo em parte
    . == 5 ~ 4,          # Concordo totalmente
    TRUE ~ NA_real_
  ))) %>%
  as.data.frame() %>%
  na.omit()
```

3. Rodando o Modelo de Créditos Parciais (PCM)

```
pcm_A_rev <- PCM(data_A_revisao)
```

```
## Warning:
```

```
## The following items have no 0-responses:
```

```
## BFI02 BFI07 BFI12 BFI17 BFI22 BFI27 BFI32 BFI37 BFI42 BFI47 BFI52 BFI57
```

```
## Responses are shifted such that lowest category is 0.
```

4. Verificando os Thresholds (O seu objetivo agora)

```
thres_A_rev <- thresholds(pcm_A_rev)
print(thres_A_rev$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3
## BFI02  0.39233331 -1.3414463  0.4175249  2.1009213
## BFI07 -1.05143873 -2.5539357 -0.1436550 -0.4567255
## BFI12  0.81012553 -1.0635226  2.1387720  1.3551272
## BFI17  0.21870514 -0.0833026  0.3566346  0.3827834
## BFI22  0.60495157 -1.1241388  1.6718563  1.2671372
## BFI27  0.46493695 -0.9765910  0.8708101  1.5005918
## BFI32 -0.69583245 -3.0285690 -0.1201235  1.0611951
## BFI37  0.78322683 -1.1983762  1.9936510  1.5544057
## BFI42  1.65339753 -0.1362727  2.7799078  2.3165575
## BFI47  0.92843024 -0.6723492  1.8411062  1.6165337
## BFI52 -0.08935583 -0.4350722  0.2563605      NA
## BFI57  0.67702503 -1.3122584  1.4312414  1.9120920
```

```
data_A_revisao <- data_A_revisao[c(1,6,7,12)]

# 5. Verificando o Item Fit (Para ver quem "ajusta" ao traço)
pcm_A_rev <- PCM(data_A_revisao)
```

```
## Warning:
## The following items have no 0-responses:
## BFI02 BFI27 BFI32 BFI57
## Responses are shifted such that lowest category is 0.
```

```
pp_A_rev <- person.parameter(pcm_A_rev)
fit_A_rev <- itemfit(pp_A_rev)
print(fit_A_rev)
```

```
##
## Itemfit Statistics:
##      Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI02 161.583 206  0.990    0.781  0.768  -2.553  -2.701  0.508
## BFI27 184.912 206  0.852    0.893  0.879  -1.094  -1.372  0.394
## BFI32 176.994 206  0.929    0.855  0.840  -1.390  -1.816  0.459
## BFI57 150.266 206  0.999    0.726  0.728  -3.106  -3.241  0.530
```

```
SepRel(pp_A_rev)
```

```
## Separation Reliability: 0.5782
```

```
data_C <- na.omit(data_clean[, C_items])
pcm_C <- PCM(data_C)
```

3.2.2.3 Conscienciosidade

```
## Warning:
## The following items have no 0-responses:
## BFI03 BFI08 BFI13 BFI18 BFI23 BFI28 BFI33 BFI38 BFI43 BFI48 BFI53 BFI58
## Responses are shifted such that lowest category is 0.
```

```
thres_C <- thresholds(pcm_C)
print(thres_C$threshtable[[1]])
```

```
##      Location  Threshold 1 Threshold 2 Threshold 3 Threshold 4
## BFI03 0.65001031 -0.006016892  1.6347928 -0.03174116  1.0030065
## BFI08 0.47502056 -0.505312676  0.9654202  0.49696217  0.9430125
## BFI13 -0.47943151 -1.299559634 -0.2481541 -0.56587380  0.1958615
## BFI18 0.35748201 -0.609417501  0.3091322  0.22049668  1.5097167
## BFI23 0.59083332 -0.358185014  1.2592042  0.66518965  0.7971244
## BFI28 2.15434630  1.228011098  2.6844380  2.55058978      NA
```

```
## BFI33 0.04237849 -1.313258267 0.2269466 0.09314407 1.1626816
## BFI38 -0.35264737 -1.537796790 -0.2069569 -0.56156247 0.8957267
## BFI43 -0.39962188 0.017541192 -1.0236597 -0.19274714 NA
## BFI48 -0.07253385 -0.996534697 0.2659572 0.23497465 0.2054675
## BFI53 -0.28234414 -1.113586197 -0.4247270 -0.07496210 0.4838987
## BFI58 0.36504367 -0.306449485 0.3380291 0.83113321 0.5974618
```

```
# 1. Recodificando: unindo apenas 2 e 3
```

```
# 1 -> 1 (Discordo totalmente)
```

```
# 2 e 3 -> 2 (Não concordo / Neutro)
```

```
# 4 -> 3 (Concordo em parte)
```

```
# 5 -> 4 (Concordo totalmente)
```

```
data_C_4cat <- data_clean %>%
  select(all_of(C_items)) %>%
  mutate(across(everything(), ~ case_when(
    . == 1 ~ 1,
    . %in% c(2, 3) ~ 2,
    . == 4 ~ 3,
    . == 5 ~ 4
  )))
```

```
# 2. Rodando o PCM (Créditos Parciais) na nova escala
```

```
# Importante: eRm exige data.frame sem NAs para o person.parameter posterior
```

```
data_C_4_clean <- as.data.frame(na.omit(data_C_4cat))
```

```
pcm_C_4 <- PCM(data_C_4_clean)
```

```
## Warning:
```

```
## The following items have no 0-responses:
```

```
## BFI03 BFI08 BFI13 BFI18 BFI23 BFI28 BFI33 BFI38 BFI43 BFI48 BFI53 BFI58
```

```
## Responses are shifted such that lowest category is 0.
```

```
# 3. Verificando os Thresholds (Agora devem ser T1 < T2 < T3)
```

```
thres_C_4 <- thresholds(pcm_C_4)
```

```
print(thres_C_4$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3
## BFI03 0.96938168 -0.03064595 1.50280838 1.4359826
## BFI08 0.74954582 -0.74849340 1.61725518 1.3798757
## BFI13 -0.49560992 -2.08229444 0.04336887 0.5520958
## BFI18 0.61882984 -1.14770755 1.04169299 1.9625041
## BFI23 0.89382972 -0.49780863 1.94437321 1.2349246
## BFI28 2.99252550 1.42345179 4.56159921 NA
## BFI33 0.19737342 -1.87890863 0.88041569 1.5906132
## BFI38 -0.31721813 -2.31014390 0.07008387 1.2884056
## BFI43 -0.08614281 -0.31152762 0.13924200 NA
## BFI48 0.02384912 -1.54196975 1.02749451 0.5860226
## BFI53 -0.23127582 -2.04602312 0.48573628 0.8664594
## BFI58 0.61215025 -0.84192818 1.65136055 1.0270184
```

```
# 4. Verificando Item Fit
```

```
pp_C_4 <- person.parameter(pcm_C_4)
```

```
fit_C_4 <- itemfit(pp_C_4)
```

```
print(fit_C_4)
```

```
##
## Itemfit Statistics:
##      Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI03 148.667 215  1.000    0.688    0.716   -3.456   -3.657    0.694
## BFI08 160.724 215  0.998    0.744    0.783   -2.760   -2.691    0.649
## BFI13 199.067 215  0.775    0.922    0.991   -0.546   -0.057    0.434
## BFI18 219.287 215  0.406    1.015    1.011    0.196    0.153    0.463
## BFI23 204.118 215  0.692    0.945    0.960   -0.502   -0.431    0.531
## BFI28 415.729 215  0.000    1.925    1.749    8.556    7.510   -0.430
## BFI33 140.041 215  1.000    0.648    0.679   -4.255   -4.300    0.734
## BFI38 180.907 215  0.956    0.838    0.842   -1.666   -1.868    0.572
## BFI43 236.590 215  0.149    1.095    1.071    0.625    0.702    0.316
## BFI48 143.015 215  1.000    0.662    0.762   -2.944   -3.020    0.635
## BFI53 189.349 215  0.896    0.877    0.819   -1.064   -2.198    0.593
## BFI58 217.054 215  0.448    1.005    1.000    0.081    0.025    0.489
```

```
# Seleccionando os 5 sobreviventes
items_C_final <- paste0("BFI", sprintf("%02d", c(13, 18, 33, 38, 53)))
data_C_final <- data_C_4_clean[, items_C_final]

# Teste final de Rasch apenas com eles
pcm_C_final <- PCM(data_C_final)
```

```
## Warning:
## The following items have no 0-responses:
## BFI13 BFI18 BFI33 BFI38 BFI53
## Responses are shifted such that lowest category is 0.
```

```
fit_C_final <- itemfit(person.parameter(pcm_C_final))

thresholds(pcm_C_final)
```

```
##
## Design Matrix Block 1:
##      Location Threshold 1 Threshold 2 Threshold 3
## BFI13  0.08082    -1.85717    0.60939    1.49024
## BFI18  1.43270    -0.68971    1.83777    3.15005
## BFI33  0.94818    -1.47634    1.63202    2.68886
## BFI38  0.33075    -2.01868    0.71341    2.29753
## BFI53  0.41910    -1.74738    1.15128    1.85340
```

```
pp_C_4 <- person.parameter(pcm_C_final)
itemfit(pp_C_4)
```

```
##
## Itemfit Statistics:
##      Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI13 226.286 194  0.056    1.160    1.012    1.189    0.153    0.348
## BFI18 180.622 194  0.746    0.926    0.942   -0.775   -0.603    0.399
## BFI33 132.914 194  1.000    0.682    0.696   -3.695   -3.759    0.632
## BFI38 170.372 194  0.888    0.874    0.829   -1.273   -1.904    0.518
## BFI53 146.793 194  0.995    0.753    0.729   -2.301   -3.164    0.630
```

```
data_N <- na.omit(data_clean[, N_items])
pcm_N <- PCM(data_N)
```

3.2.2.4 Neuroticismo

```
## Warning:
## The following items have no 0-responses:
## BFI04 BFI09 BFI14 BFI19 BFI24 BFI29 BFI34 BFI39 BFI44 BFI49 BFI54 BFI59
## Responses are shifted such that lowest category is 0.
```

```
thres_N <- thresholds(pcm_N)
print(thres_N$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3 Threshold 4
## BFI04  0.08120770 -1.6746946  0.7505767  0.08006918  1.1688795
## BFI09  0.50983643 -0.9137862  0.7764446  0.55111594  1.6255714
## BFI14  0.66955989 -0.2202411  0.6025531  0.44886782  1.8470598
## BFI19 -0.08793771 -0.6828845 -0.5668365 -0.52298947  1.4209596
## BFI24  1.41371803 -0.1746659  1.2413123  0.79430841  3.7939173
## BFI29  0.39622994 -0.6719029  0.5476074  0.40228684  1.3069284
## BFI34 -1.32713469 -3.6513819 -0.7644926 -1.00787738  0.1152132
## BFI39  1.28932369  0.5293767  1.1525914  1.37776776  2.0975589
## BFI44  0.77348836 -0.8941183  0.7663192  1.64058783  1.5811648
## BFI49 -0.37800930 -0.9631433 -0.4969426 -0.57816787  0.5262165
## BFI54  0.65763960  0.2958254  0.3555869  0.23788375  1.7412623
## BFI59  0.06678807 -0.2786882 -0.3131872  0.11515568  0.7438720
```

1. Aplicando a junção de 2 e 3 para o Neuroticismo

```
data_N_4cat <- data_clean %>%
  select(all_of(N_items)) %>%
  mutate(across(everything(), ~ case_when(
    . == 1 ~ 1,
    . %in% c(2, 3) ~ 2,
    . == 4 ~ 3,
    . == 5 ~ 4
  )))
```

2. Rodando o PCM na nova escala

```
data_N_4_clean <- as.data.frame(na.omit(data_N_4cat))
pcm_N_4 <- PCM(data_N_4_clean)
```

```
## Warning:
## The following items have no 0-responses:
## BFI04 BFI09 BFI14 BFI19 BFI24 BFI29 BFI34 BFI39 BFI44 BFI49 BFI54 BFI59
## Responses are shifted such that lowest category is 0.
```

3. Verificando os novos Thresholds

```
thres_N_4 <- thresholds(pcm_N_4)
print(thres_N_4$threshtable[[1]])
```



```
##           Location Threshold 1 Threshold 2 Threshold 3
## BFI04  0.14956637 -2.1141140  1.17171529  1.3910979
## BFI09  0.73229942 -1.3199945  1.62444603  1.8924467
## BFI14  0.96154260 -0.6636172  1.41746241  2.1307826
## BFI19 -0.01171352 -1.6415205 -0.03765347  1.6440334
## BFI24  1.93681999 -0.4309441  2.10642880  4.1349753
## BFI29  0.58959488 -1.1437239  1.35499469  1.5575139
## BFI34 -1.70161899 -4.7304346 -0.61972921  0.2453069
## BFI39  1.76701290  0.2630528  2.59873454  2.4392513
## BFI44  1.08483843 -1.3060795  2.67220949  1.8883853
## BFI49 -0.42097092 -1.8872465 -0.07510371  0.6994375
## BFI54  0.95479546 -0.2331357  1.07681891  2.0207031
## BFI59  0.17733934 -1.1069093  0.68049239  0.9584350
```

```
# 1. Criando o objeto com os 9 sobreviventes
items_N_final <- paste0("BFI", sprintf("%02d", c(04, 09, 14, 19, 24, 29, 49, 54, 59)))
data_N_final  <- data_N_4_clean[, items_N_final]

# 2. Rodando o Modelo Rasch Final
pcm_N_final <- PCM(data_N_final)
```

```
## Warning:
## The following items have no 0-responses:
## BFI04 BFI09 BFI14 BFI19 BFI24 BFI29 BFI49 BFI54 BFI59
## Responses are shifted such that lowest category is 0.
```

```
pp_N <- person.parameter(pcm_N_final)
fit_N <- itemfit(pp_N)
print(fit_N)
```

```
##
## Itemfit Statistics:
##           Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI04  177.958 212   0.957    0.835  0.832  -1.654  -1.868  0.600
## BFI09  211.423 212   0.498    0.993  0.910  -0.037  -0.890  0.533
## BFI14  194.977 212   0.793    0.915  0.885  -0.847  -1.226  0.561
## BFI19  206.683 212   0.590    0.970  0.975  -0.301  -0.247  0.506
## BFI24  207.884 212   0.567    0.976  0.981  -0.221  -0.177  0.457
## BFI29  165.240 212   0.992    0.776  0.770  -2.427  -2.624  0.664
## BFI49  197.000 212   0.762    0.925  0.908  -0.770  -1.059  0.546
## BFI54  152.625 212   0.999    0.717  0.750  -3.016  -2.961  0.668
## BFI59  258.498 212   0.016    1.214  1.175   2.104   1.915  0.403
```

```
data_0 <- na.omit(data_clean[, 0_items])
pcm_0 <- PCM(data_0)
```

3.2.2.5 Abertura à experiência

```
## Warning:
```

```
## The following items have no 0-responses:
## BFI05 BFI10 BFI15 BFI20 BFI25 BFI30 BFI35 BFI40 BFI45 BFI50 BFI55 BFI60
## Responses are shifted such that lowest category is 0.
```

```
thres_0 <- thresholds(pcm_0)
print(thres_0$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3 Threshold 4
## BFI05 -0.36538621 -2.42082273  0.8370807  0.12774083 -0.005543642
## BFI10  0.06460300  1.02531924 -1.1507920  0.31928175          NA
## BFI15 -0.04923371 -1.30786082  0.3078102 -0.46205114  1.265166895
## BFI20  0.16053552 -0.06371738  0.7454029 -0.56351761  0.523974157
## BFI25  0.89959814  0.42495153  0.8447730  0.56954920  1.759118805
## BFI30  0.13060017 -0.78765516  0.6344641  0.05496537  0.620626345
## BFI35 -0.21117315 -0.31749522 -0.3871708 -0.56440322  0.424376645
## BFI40  0.41400904  0.03027370  0.2791346  0.17033237  1.176295485
## BFI45  0.10832010 -0.83645932  0.7197527  0.13458514  0.415401857
## BFI50  0.15417012 -0.19000699  0.3473055  0.33200289  0.127379065
## BFI55  0.43113793 -0.22805980  0.2545931  0.81762698  0.880391442
## BFI60  0.22376195 -0.62161168  0.1803641  0.13252518  1.203770160
```

```
library(eRm)
```

```
# 1. Recodificando Abertura (0) - Unindo 2 e 3
```

```
data_0_4cat <- data_clean %>%
  select(all_of(0_items)) %>%
  mutate(across(everything(), ~ case_when(
    . == 1 ~ 1,
    . %in% c(2, 3) ~ 2,
    . == 4 ~ 3,
    . == 5 ~ 4
  )))
```

```
# 2. Rodando o PCM na nova escala
```

```
data_0_4_clean <- as.data.frame(na.omit(data_0_4cat))
pcm_0_4 <- PCM(data_0_4_clean)
```

```
## Warning:
```

```
## The following items have no 0-responses:
## BFI05 BFI10 BFI15 BFI20 BFI25 BFI30 BFI35 BFI40 BFI45 BFI50 BFI55 BFI60
## Responses are shifted such that lowest category is 0.
```

```
# 3. Verificando os novos Thresholds
```

```
thres_0_4 <- thresholds(pcm_0_4)
print(thres_0_4$threshtable[[1]])
```

```
##           Location Threshold 1 Threshold 2 Threshold 3
## BFI05 -0.35793722 -2.68062559  1.19946893  0.4073450
## BFI10  0.38409667  0.03914378  0.72904956          NA
## BFI15  0.07857136 -1.84583937  0.37949054  1.7020629
## BFI20  0.34721713 -0.36615443  0.46350132  0.9443045
## BFI25  1.31710451  0.13809971  1.60393978  2.2092740
```

```
## BFI30  0.30699113 -1.15145736  1.02395562  1.0484751
## BFI35 -0.13852158 -1.31314466  0.05601757  0.8415623
## BFI40  0.68965757 -0.54289630  0.99233971  1.6195293
## BFI45  0.27481714 -1.15738038  1.14252448  0.8393073
## BFI50  0.33755662 -0.71359883  1.17801827  0.5482504
## BFI55  0.70971973 -0.82432213  1.62712712  1.3263542
## BFI60  0.44017560 -1.24918700  0.92323629  1.6464775

## Salvos
items_0_final <- paste0("BFI", sprintf("%02d", c(15, 20, 30, 35, 40, 60)))
data_0_final <- data_0_4_clean[, items_0_final]

# 3. Teste de Invariância (Andersen) para garantir a robustez
pcm_0_final <- PCM(data_0_final)
```

```
## Warning:
## The following items have no 0-responses:
## BFI15 BFI20 BFI30 BFI35 BFI40 BFI60
## Responses are shifted such that lowest category is 0.
```

```
pp_0 <- person.parameter(pcm_0_final)
fit_0 <- itemfit(pp_0)
print(fit_0)
```

```
##
## Itemfit Statistics:
##           Chisq  df p-value Outfit MSQ Infit MSQ Outfit t Infit t Discrim
## BFI15 163.667 194  0.945      0.839  0.787 -1.702 -2.429  0.575
## BFI20 186.581 194  0.636      0.957  0.898 -0.286 -0.978  0.463
## BFI30 190.833 194  0.551      0.979  0.974 -0.140 -0.253  0.395
## BFI35 132.424 194  1.000      0.679  0.740 -2.632 -2.602  0.596
## BFI40 226.185 194  0.057      1.160  1.133  1.532  1.401  0.257
## BFI60 154.769 194  0.983      0.794  0.791 -2.202 -2.434  0.590
```

3.2.3 BFI-2 com itens selecionados

```
library(tidyverse)
library(lavaan)

# 1. Definindo os itens finais purificados
final_E <- paste0("BFI", sprintf("%02d", c(01, 06, 21, 26, 36, 41, 56)))
final_A <- paste0("BFI", sprintf("%02d", c(02, 27, 32, 57)))
final_C <- paste0("BFI", sprintf("%02d", c(13, 18, 33, 38, 53)))
final_N <- paste0("BFI", sprintf("%02d", c(04, 09, 14, 19, 24, 29, 49, 54, 59)))
final_O <- paste0("BFI", sprintf("%02d", c(15, 20, 30, 35, 40, 60)))

all_final_items <- c(final_E, final_A, final_C, final_N, final_O)

# 2. Criando o banco definitivo (Padronização Universal 2+3)
data_bfi_final <- data_clean %>%
  mutate(across(all_of(all_final_items), ~ case_when(
```

```

. == 1 ~ 1,
. %in% c(2, 3) ~ 2,
. == 4 ~ 3,
. == 5 ~ 4,
TRUE ~ NA_real_
))) %>%
select(all_of(all_final_items))

# 3. Modelo de 5 Fatores Correlacionados
mod_bfi_final <- '
  Extro    =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56
  Agree    =~ BFI02 + BFI27 + BFI32 + BFI57
  Consc    =~ BFI13 + BFI18 + BFI33 + BFI38 + BFI53
  Neuro    =~ BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54
  OpenM    =~ BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60
'

fit_bfi_final <- cfa(mod_bfi_final, data = data_bfi_final,
  ordered = TRUE, estimator = "WLSMV", std.lv = TRUE)

# 4. Fit Measures (Os seus favoritos)
fitMeasures(fit_bfi_final, fit.measures = c("chisq", "df", "pvalue", "cfi", "tli", "rmsea",
  "rmsea.ci.lower", "rmsea.ci.upper", "srmr"))

```

##	chisq	df	pvalue	cfi	tli
##	1002.254	395.000	0.000	0.923	0.915
##	rmsea	rmsea.ci.lower	rmsea.ci.upper	srmr	
##	0.085	0.078	0.091	0.101	

```
compRelSEM(fit_bfi_final)
```

```
## Extro Agree Consc Neuro OpenM
## 0.738 0.572 0.719 0.799 0.734
```

```
summary(fit_bfi_final, standardized = T, rsquare = T)
```

```
## lavaan 0.6-20 ended normally after 26 iterations
##
##      Estimator                      DWLS
##      Optimization method           NLMINB
##      Number of model parameters      130
##
##      Number of observations          216
##
## Model Test User Model:
##
##      Standard      Scaled
##      Test Statistic 1002.254 893.535
##      Degrees of freedom      395      395
##      P-value (Chi-square)    0.000    0.000
##      Scaling correction factor      1.462
##      Shift parameter          208.192
##      simple second-order correction
```

```

##
## Parameter Estimates:
##
##   Parameterization          Delta
##   Standard errors          Robust.sem
##   Information              Expected
##   Information saturated (h1) model    Unstructured
##
## Latent Variables:
##           Estimate  Std.Err  z-value  P(>|z|)  Std.lv  Std.all
## Extro =~
##   BFI01           0.566    0.052   10.831    0.000    0.566    0.566
##   BFI06           0.672    0.049   13.697    0.000    0.672    0.672
##   BFI21           0.665    0.047   14.018    0.000    0.665    0.665
##   BFI26           0.457    0.063    7.303    0.000    0.457    0.457
##   BFI36           0.318    0.068    4.675    0.000    0.318    0.318
##   BFI41           0.748    0.036   20.582    0.000    0.748    0.748
##   BFI56           0.590    0.049   12.005    0.000    0.590    0.590
## Agree =~
##   BFI02           0.366    0.067    5.469    0.000    0.366    0.366
##   BFI27           0.421    0.069    6.073    0.000    0.421    0.421
##   BFI32           0.907    0.069   13.116    0.000    0.907    0.907
##   BFI57           0.640    0.059   10.906    0.000    0.640    0.640
## Consc =~
##   BFI13           0.688    0.062   11.117    0.000    0.688    0.688
##   BFI18           0.381    0.059    6.443    0.000    0.381    0.381
##   BFI33           0.601    0.056   10.804    0.000    0.601    0.601
##   BFI38           0.802    0.042   19.207    0.000    0.802    0.802
##   BFI53           0.785    0.043   18.369    0.000    0.785    0.785
## Neuro =~
##   BFI04           0.717    0.062   11.569    0.000    0.717    0.717
##   BFI09           0.704    0.058   12.072    0.000    0.704    0.704
##   BFI14           0.578    0.062    9.341    0.000    0.578    0.578
##   BFI19           0.478    0.061    7.812    0.000    0.478    0.478
##   BFI24           0.708    0.055   12.789    0.000    0.708    0.708
##   BFI29           0.784    0.042   18.488    0.000    0.784    0.784
##   BFI49           0.527    0.062    8.521    0.000    0.527    0.527
##   BFI54           0.667    0.052   12.708    0.000    0.667    0.667
## OpenM =~
##   BFI15           0.846    0.052   16.145    0.000    0.846    0.846
##   BFI20           0.498    0.060    8.338    0.000    0.498    0.498
##   BFI30           0.517    0.066    7.790    0.000    0.517    0.517
##   BFI35           0.684    0.059   11.620    0.000    0.684    0.684
##   BFI40           0.353    0.075    4.685    0.000    0.353    0.353
##   BFI60           0.850    0.047   18.068    0.000    0.850    0.850
##
## Covariances:
##           Estimate  Std.Err  z-value  P(>|z|)  Std.lv  Std.all
## Extro ~~
##   Agree           0.677    0.063   10.675    0.000    0.677    0.677
##   Consc           0.817    0.043   18.787    0.000    0.817    0.817
##   Neuro          -0.637    0.049  -12.939    0.000   -0.637   -0.637
##   OpenM           0.656    0.055   11.874    0.000    0.656    0.656
## Agree ~~

```

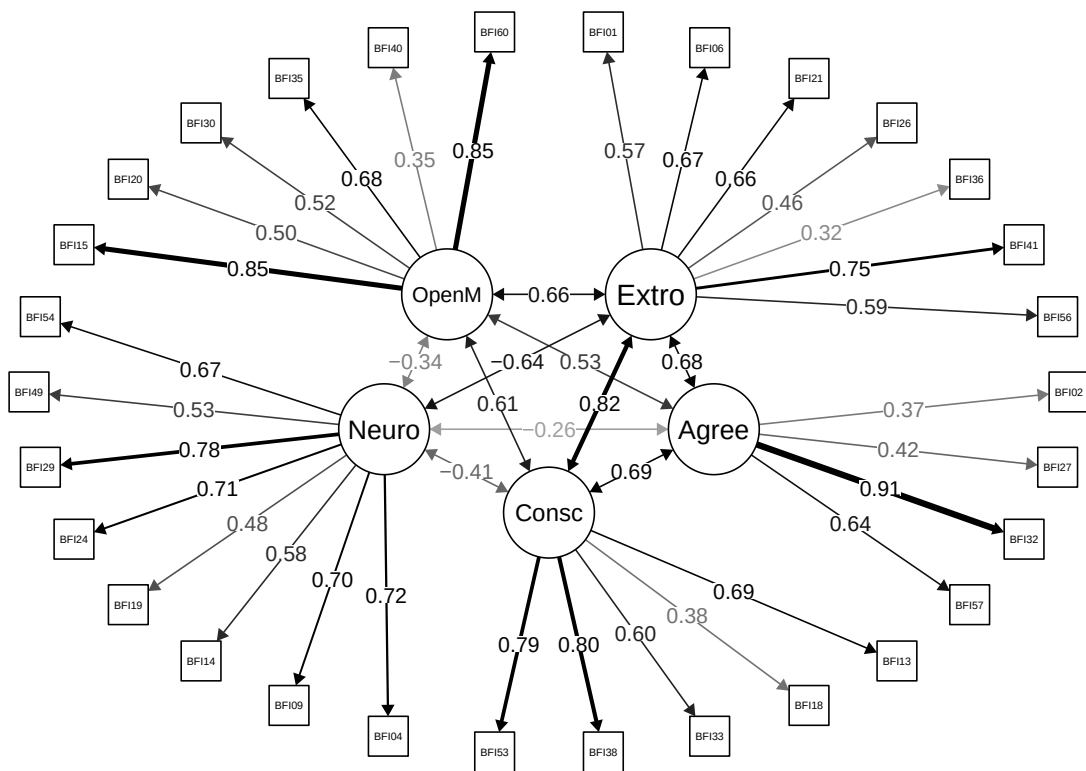
##	Consc	0.686	0.059	11.656	0.000	0.686	0.686
##	Neuro	-0.259	0.078	-3.332	0.001	-0.259	-0.259
##	OpenM	0.534	0.067	7.966	0.000	0.534	0.534
##	Consc ~~						
##	Neuro	-0.412	0.070	-5.918	0.000	-0.412	-0.412
##	OpenM	0.608	0.055	11.147	0.000	0.608	0.608
##	Neuro ~~						
##	OpenM	-0.342	0.065	-5.256	0.000	-0.342	-0.342
##							
##	Thresholds:						
##		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
##	BFI01 t1	-2.602	0.343	-7.589	0.000	-2.602	-2.602
##	BFI01 t2	-0.967	0.102	-9.510	0.000	-0.967	-0.967
##	BFI01 t3	0.175	0.086	2.036	0.042	0.175	0.175
##	BFI06 t1	-2.200	0.225	-9.771	0.000	-2.200	-2.200
##	BFI06 t2	-0.482	0.089	-5.407	0.000	-0.482	-0.482
##	BFI06 t3	0.631	0.092	6.866	0.000	0.631	0.631
##	BFI21 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
##	BFI21 t2	-0.222	0.086	-2.578	0.010	-0.222	-0.222
##	BFI21 t3	0.719	0.094	7.647	0.000	0.719	0.719
##	BFI26 t1	-1.516	0.133	-11.419	0.000	-1.516	-1.516
##	BFI26 t2	-0.294	0.087	-3.389	0.001	-0.294	-0.294
##	BFI26 t3	0.443	0.089	5.006	0.000	0.443	0.443
##	BFI36 t1	-1.383	0.123	-11.249	0.000	-1.383	-1.383
##	BFI36 t2	0.210	0.086	2.442	0.015	0.210	0.210
##	BFI36 t3	1.150	0.110	10.499	0.000	1.150	1.150
##	BFI41 t1	-1.593	0.139	-11.436	0.000	-1.593	-1.593
##	BFI41 t2	-0.258	0.087	-2.984	0.003	-0.258	-0.258
##	BFI41 t3	0.603	0.091	6.603	0.000	0.603	0.603
##	BFI56 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI56 t2	-0.234	0.086	-2.713	0.007	-0.234	-0.234
##	BFI56 t3	0.689	0.093	7.388	0.000	0.689	0.689
##	BFI02 t1	-1.847	0.167	-11.090	0.000	-1.847	-1.847
##	BFI02 t2	-0.576	0.091	-6.338	0.000	-0.576	-0.576
##	BFI02 t3	0.719	0.094	7.647	0.000	0.719	0.719
##	BFI27 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
##	BFI27 t2	-0.418	0.088	-4.737	0.000	-0.418	-0.418
##	BFI27 t3	0.495	0.089	5.541	0.000	0.495	0.495
##	BFI32 t1	-2.602	0.343	-7.589	0.000	-2.602	-2.602
##	BFI32 t2	-1.085	0.107	-10.187	0.000	-1.085	-1.085
##	BFI32 t3	0.070	0.086	0.815	0.415	0.070	0.070
##	BFI57 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
##	BFI57 t2	-0.058	0.086	-0.679	0.497	-0.058	-0.058
##	BFI57 t3	0.828	0.097	8.537	0.000	0.828	0.828
##	BFI13 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI13 t2	-1.106	0.107	-10.293	0.000	-1.106	-1.106
##	BFI13 t3	-0.210	0.086	-2.442	0.015	-0.210	-0.210
##	BFI18 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
##	BFI18 t2	-0.282	0.087	-3.254	0.001	-0.282	-0.282
##	BFI18 t3	0.674	0.093	7.258	0.000	0.674	0.674
##	BFI33 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI33 t2	-0.456	0.089	-5.140	0.000	-0.456	-0.456
##	BFI33 t3	0.456	0.089	5.140	0.000	0.456	0.456
##	BFI38 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355

##	BFI38 t2	-0.931	0.100	-9.273	0.000	-0.931	-0.931
##	BFI38 t3	0.175	0.086	2.036	0.042	0.175	0.175
##	BFI53 t1	-2.200	0.225	-9.771	0.000	-2.200	-2.200
##	BFI53 t2	-0.812	0.097	-8.412	0.000	-0.812	-0.812
##	BFI53 t3	0.023	0.085	0.272	0.786	0.023	0.023
##	BFI04 t1	-1.325	0.119	-11.116	0.000	-1.325	-1.325
##	BFI04 t2	0.343	0.087	3.929	0.000	0.343	0.343
##	BFI04 t3	1.085	0.107	10.187	0.000	1.085	1.085
##	BFI09 t1	-0.913	0.100	-9.153	0.000	-0.913	-0.913
##	BFI09 t2	0.660	0.093	7.128	0.000	0.660	0.660
##	BFI09 t3	1.414	0.125	11.305	0.000	1.414	1.414
##	BFI14 t1	-0.617	0.092	-6.734	0.000	-0.617	-0.617
##	BFI14 t2	0.646	0.092	6.997	0.000	0.646	0.646
##	BFI14 t3	1.480	0.130	11.392	0.000	1.480	1.480
##	BFI19 t1	-1.271	0.116	-10.960	0.000	-1.271	-1.271
##	BFI19 t2	-0.175	0.086	-2.036	0.042	-0.175	-0.175
##	BFI19 t3	0.967	0.102	9.510	0.000	0.967	0.967
##	BFI24 t1	-0.431	0.088	-4.871	0.000	-0.431	-0.431
##	BFI24 t2	1.085	0.107	10.187	0.000	1.085	1.085
##	BFI24 t3	2.355	0.262	8.984	0.000	2.355	2.355
##	BFI29 t1	-0.862	0.098	-8.786	0.000	-0.862	-0.862
##	BFI29 t2	0.508	0.090	5.674	0.000	0.508	0.508
##	BFI29 t3	1.221	0.113	10.786	0.000	1.221	1.221
##	BFI49 t1	-1.414	0.125	-11.305	0.000	-1.414	-1.414
##	BFI49 t2	-0.331	0.087	-3.795	0.000	-0.331	-0.331
##	BFI49 t3	0.522	0.090	5.807	0.000	0.522	0.522
##	BFI54 t1	-0.456	0.089	-5.140	0.000	-0.456	-0.456
##	BFI54 t2	0.535	0.090	5.940	0.000	0.535	0.535
##	BFI54 t3	1.383	0.123	11.249	0.000	1.383	1.383
##	BFI15 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI15 t2	-0.931	0.100	-9.273	0.000	-0.931	-0.931
##	BFI15 t3	0.222	0.086	2.578	0.010	0.222	0.222
##	BFI20 t1	-1.847	0.167	-11.090	0.000	-1.847	-1.847
##	BFI20 t2	-0.986	0.102	-9.627	0.000	-0.986	-0.986
##	BFI20 t3	-0.140	0.086	-1.629	0.103	-0.140	-0.140
##	BFI30 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI30 t2	-0.734	0.094	-7.776	0.000	-0.734	-0.734
##	BFI30 t3	0.000	0.085	0.000	1.000	0.000	0.000
##	BFI35 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI35 t2	-1.271	0.116	-10.960	0.000	-1.271	-1.271
##	BFI35 t3	-0.270	0.087	-3.119	0.002	-0.270	-0.270
##	BFI40 t1	-1.682	0.148	-11.381	0.000	-1.682	-1.682
##	BFI40 t2	-0.576	0.091	-6.338	0.000	-0.576	-0.576
##	BFI40 t3	0.294	0.087	3.389	0.001	0.294	0.294
##	BFI60 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI60 t2	-0.646	0.092	-6.997	0.000	-0.646	-0.646
##	BFI60 t3	0.282	0.087	3.254	0.001	0.282	0.282
##							
##	Variances:						
##		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
##	.BFI01	0.679				0.679	0.679
##	.BFI06	0.548				0.548	0.548
##	.BFI21	0.558				0.558	0.558
##	.BFI26	0.791				0.791	0.791

##	.BFI36	0.899	0.899	0.899
##	.BFI41	0.441	0.441	0.441
##	.BFI56	0.652	0.652	0.652
##	.BFI02	0.866	0.866	0.866
##	.BFI27	0.823	0.823	0.823
##	.BFI32	0.177	0.177	0.177
##	.BFI57	0.590	0.590	0.590
##	.BFI13	0.527	0.527	0.527
##	.BFI18	0.855	0.855	0.855
##	.BFI33	0.639	0.639	0.639
##	.BFI38	0.356	0.356	0.356
##	.BFI53	0.383	0.383	0.383
##	.BFI04	0.485	0.485	0.485
##	.BFI09	0.504	0.504	0.504
##	.BFI14	0.666	0.666	0.666
##	.BFI19	0.772	0.772	0.772
##	.BFI24	0.499	0.499	0.499
##	.BFI29	0.386	0.386	0.386
##	.BFI49	0.722	0.722	0.722
##	.BFI54	0.555	0.555	0.555
##	.BFI15	0.284	0.284	0.284
##	.BFI20	0.752	0.752	0.752
##	.BFI30	0.733	0.733	0.733
##	.BFI35	0.532	0.532	0.532
##	.BFI40	0.875	0.875	0.875
##	.BFI60	0.278	0.278	0.278
##	Extro	1.000	1.000	1.000
##	Agree	1.000	1.000	1.000
##	Consc	1.000	1.000	1.000
##	Neuro	1.000	1.000	1.000
##	OpenM	1.000	1.000	1.000
##				
##	R-Square:			
##		Estimate		
##	BFI01	0.321		
##	BFI06	0.452		
##	BFI21	0.442		
##	BFI26	0.209		
##	BFI36	0.101		
##	BFI41	0.559		
##	BFI56	0.348		
##	BFI02	0.134		
##	BFI27	0.177		
##	BFI32	0.823		
##	BFI57	0.410		
##	BFI13	0.473		
##	BFI18	0.145		
##	BFI33	0.361		
##	BFI38	0.644		
##	BFI53	0.617		
##	BFI04	0.515		
##	BFI09	0.496		
##	BFI14	0.334		
##	BFI19	0.228		


```
##      BFI24      0.501
##      BFI29      0.614
##      BFI49      0.278
##      BFI54      0.445
##      BFI15      0.716
##      BFI20      0.248
##      BFI30      0.267
##      BFI35      0.468
##      BFI40      0.125
##      BFI60      0.722
```

```
library(semPlot)
semplot_modelo <- semPaths(fit_bfi_final,"std",layout="circle",residuals=F,sizeLat=8,sizeLat2=8,edge.co
                           mar=c(2, 2, 2, 2),esize=4,curvePivot = F, intercepts=F,thresholds = F,
                           nCharNodes=0,sizeMan=3.5, edge.label.position=0.5)
```



3.2.4 BFI-2 bifactor campeã

```
# --- MODELO B: BIFACTOR (GFP e específicos competindo pelos itens) ---
# Nota: Todos os fatores devem ser ortogonais no Bifactor puro
mod_bfi_bifactor <- '
  GFP =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56 +
         BFI02 + BFI27 + BFI32 + BFI57 +
```

```

      BFI13 + BFI18 + BFI33 + BFI38 + BFI53 +
      BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54 +
      BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60

E_s =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56
A_s =~ BFI02 + BFI27 + BFI32 + BFI57
C_s =~ BFI13 + BFI18 + BFI33 + BFI38 + BFI53
N_s =~ BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54
O_s =~ BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60
,

fit_bfi_bifactor <- cfa(mod_bfi_bifactor, data = data_bfi_final,
      ordered = TRUE, estimator = "WLSMV", std.lv = TRUE,
      orthogonal = TRUE)

fitMeasures(fit_bfi_bifactor, fit.measures = c("chisq","df","pvalue","cfi","tli", "rmsea",
      "rmsea.ci.lower", "rmsea.ci.upper","srmr"))

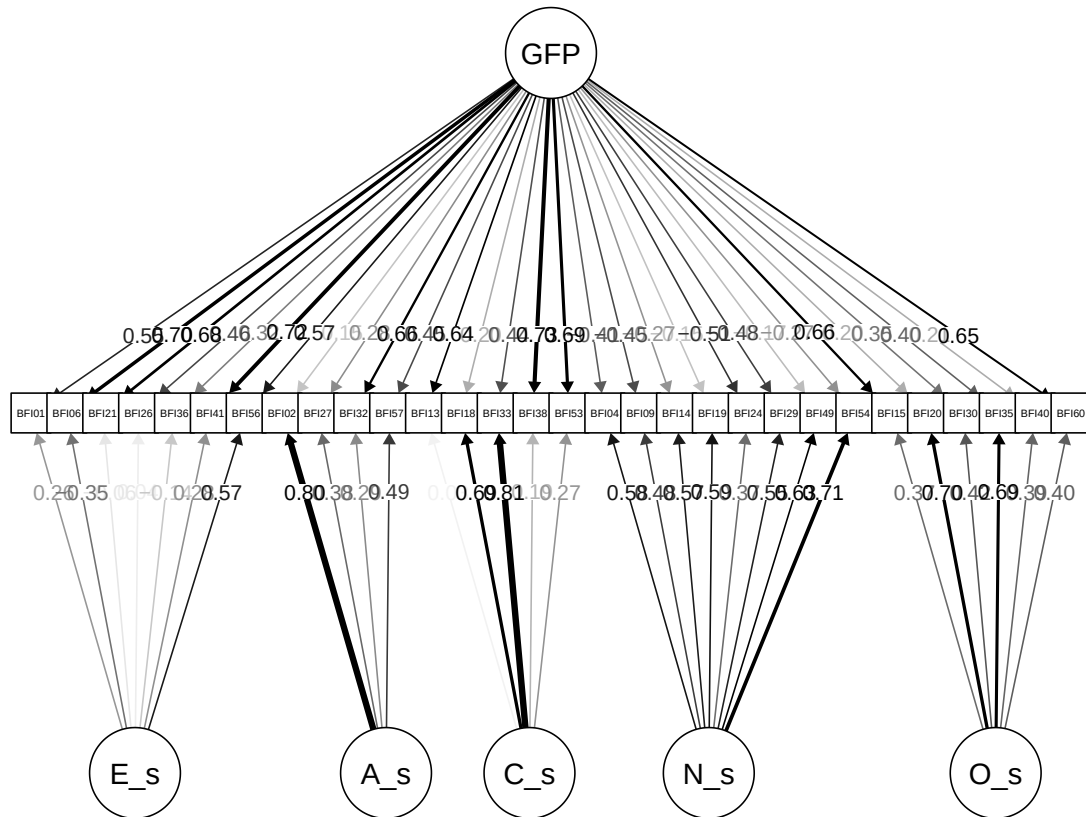
##          chisq          df          pvalue          cfi          tli
##      675.533      375.000          0.000      0.962      0.956
##      rmsea rmsea.ci.lower rmsea.ci.upper          srmr
##      0.061          0.054          0.068      0.087

compRelSEM(fit_bfi_bifactor)

##   GFP   E_s   A_s   C_s   N_s   O_s
## 0.495 0.017 0.423 0.304 0.627 0.451

semPaths(fit_bfi_bifactor,"std",layout="tree2",residuals=F,sizeLat=8,sizeLat2=8,
      edge.color="black",edge.label.cex=0.8, mar=c(2, 2, 2, 2),esize=4,
      curvePivot = F, intercepts=F,thresholds = F, nCharNodes=0,sizeMan=3.5,
      edge.label.position=0.8, bifactor = "GFP")

```



3.2.5 DeYoung BFI-2

```
# --- MODELO C: SEGUNDA ORDEM (DeYoung) ---
mod_bfi_plast_est <- '
  Extraversion      =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56
  Agreeableness     =~ BFI02 + BFI27 + BFI32 + BFI57
  Conscientiousness =~ BFI13 + BFI18 + BFI33 + BFI38 + BFI53
  Negative_Emot     =~ BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54
  Open_Mindedness   =~ BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60

  Plasticity =~ Extraversion + Open_Mindedness
  Stability  =~ Agreeableness + Conscientiousness + Negative_Emot
'

fit_bfi_2order <- cfa(mod_bfi_plast_est, data = data_bfi_final,
                      ordered = TRUE, estimator = "WLSMV", std.lv = TRUE)

## Warning: lavaan->lav_object_post_check():
##   covariance matrix of latent variables is not positive definite ; use
##   lavInspect(fit, "cov.lv") to investigate.
```

```
fitMeasures(fit_bfi_2order, fit.measures = c("chisq","df","pvalue","cfi","tli", "rmsea",
                                             "rmsea.ci.lower", "rmsea.ci.upper","srmr"))
```

```
## Warning: lavaan->lav_object_post_check():
##   covariance matrix of latent variables is not positive definite ; use
##   lavInspect(fit, "cov.lv") to investigate.
## Warning: lavaan->lav_object_post_check():
##   covariance matrix of latent variables is not positive definite ; use
##   lavInspect(fit, "cov.lv") to investigate.
```

	chisq	df	pvalue	cfi	tli
	1044.788	399.000	0.000	0.918	0.911
	rmsea	rmsea.ci.lower	rmsea.ci.upper	srmr	
	0.087	0.080	0.093	0.103	

```
compRelSEM(fit_bfi_2order, higher = c("Plasticity","Stability"))
```

	Extraversion	Agreeableness	Conscientiousness	Negative_Emot
	0.737	0.570	0.718	0.797
	Open_Mindedness	Plasticity	Stability	
	0.732	0.703	0.035	

3.2.6 BFI-2 DeYoung apenas com Plasticidade de segunda ordem

```
mod_bfi_plast_est <- '
  Extraversion      =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56
  Agreeableness     =~ BFI02 + BFI27 + BFI32 + BFI57
  Conscientiousness =~ BFI13 + BFI18 + BFI33 + BFI38 + BFI53
  Negative_Emot     =~ BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54
  Open_Mindedness   =~ BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60

  Plasticity =~ Extraversion + Open_Mindedness
'
```

```
fit_bfi_2order <- cfa(mod_bfi_plast_est, data = data_bfi_final,
                      ordered = TRUE, estimator = "WLSMV", std.lv = TRUE)
```

```
fitMeasures(fit_bfi_2order, fit.measures = c("chisq","df","pvalue","cfi","tli", "rmsea",
                                             "rmsea.ci.lower", "rmsea.ci.upper","srmr"))
```

	chisq	df	pvalue	cfi	tli
	1016.171	397.000	0.000	0.922	0.914
	rmsea	rmsea.ci.lower	rmsea.ci.upper	srmr	
	0.085	0.079	0.092	0.102	

```
compRelSEM(fit_bfi_2order, higher = c("Plasticity"))
```

```
##      Extraversion      Agreeableness Conscientiousness      Negative_Emot
##      0.738              0.573              0.720              0.797
##      Open_Mindedness      Plasticity
##      0.730              0.704
```

```
#semPaths(fit_bfi_2order,"std",layout="circle2",residuals=F,sizeLat=8,sizeLat2=8,
#          edge.color="black",edge.label.cex=0.8, mar=c(2, 2, 2, 2),esize=4,
#          curvePivot = F, intercepts=F,thresholds = F, nCharNodes=0,sizeMan=3.5,
#          edge.label.position=0.8, bifactor = "Plasticity")
```

3.2.7 BFI-2 bifactor com Estabilidade Emocional (neuroticismo invertido)

```
# 1. Invertendo Neuroticismo para se tornar "Estabilidade Emocional"
# 4-cat: 1->4, 2->3, 3->2, 4->1
data_bfi_final <- data_bfi_final %>%
  mutate(across(all_of(final_N), ~ 5 - .x))

# 2. Definindo o Modelo Unidimensional de Segunda Ordem (O "Big One")
mod_gfp_ortho <- '
  Extraversion      =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56
  Agreeableness      =~ BFI02 + BFI27 + BFI32 + BFI57
  Conscientiousness =~ BFI13 + BFI18 + BFI33 + BFI38 + BFI53
  Emotional_Stab      =~ BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54
  Open_Mindedness    =~ BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60

  GFP =~ BFI01 + BFI06 + BFI21 + BFI26 + BFI36 + BFI41 + BFI56 +
    BFI02 + BFI27 + BFI32 + BFI57 +
    BFI13 + BFI18 + BFI33 + BFI38 + BFI53 +
    BFI04 + BFI09 + BFI14 + BFI19 + BFI24 + BFI29 + BFI49 + BFI54 +
    BFI15 + BFI20 + BFI30 + BFI35 + BFI40 + BFI60
'

fit_gfp_ortho <- cfa(mod_gfp_ortho, data = data_bfi_final, orthogonal = TRUE,
  ordered = TRUE, estimator = "WLSMV", std.lv = TRUE)

# 3. Fit Measures
fitMeasures(fit_gfp_ortho, fit.measures = c("chisq","df","pvalue","cfi","tli", "rmsea",
  "rmsea.ci.lower", "rmsea.ci.upper","srmr"))
```

```
##      chisq      df      pvalue      cfi      tli
##      675.533    375.000      0.000    0.962    0.956
##      rmsea rmsea.ci.lower rmsea.ci.upper      srmr
##      0.061      0.054      0.068      0.087
```

```
# 4. Confiabilidade do Fator Geral
compRelSEM(fit_gfp_ortho)
```

```
##      Extraversion      Agreeableness Conscientiousness      Emotional_Stab
##      0.017              0.423              0.304              0.627
##      Open_Mindedness      GFP
##      0.451              0.757
```

```
summary(fit_gfp_ortho, standardized = T)
```

```
## lavaan 0.6-20 ended normally after 49 iterations
##
##      Estimator                      DWLS
##      Optimization method          NLMINB
##      Number of model parameters    150
##
##      Number of observations        216
##
## Model Test User Model:
##
##              Standard      Scaled
##      Test Statistic      675.533  679.958
##      Degrees of freedom      375      375
##      P-value (Chi-square)    0.000    0.000
##      Scaling correction factor      1.400
##      Shift parameter      197.290
##      simple second-order correction
##
## Parameter Estimates:
##
##      Parameterization          Delta
##      Standard errors          Robust.sem
##      Information          Expected
##      Information saturated (h1) model      Unstructured
##
## Latent Variables:
##
##              Estimate  Std.Err  z-value  P(>|z|)  Std.lv  Std.all
##      Extraversion =~
##      BFI01          0.258   0.083   3.102   0.002   0.258   0.258
##      BFI06         -0.353   0.107  -3.309   0.001  -0.353  -0.353
##      BFI21         -0.061   0.076  -0.807   0.419  -0.061  -0.061
##      BFI26          0.045   0.084   0.532   0.595   0.045   0.045
##      BFI36         -0.136   0.092  -1.480   0.139  -0.136  -0.136
##      BFI41          0.277   0.074   3.727   0.000   0.277   0.277
##      BFI56          0.574   0.112   5.114   0.000   0.574   0.574
##      Agreeableness =~
##      BFI02          0.805   0.097   8.266   0.000   0.805   0.805
##      BFI27          0.375   0.069   5.441   0.000   0.375   0.375
##      BFI32          0.289   0.083   3.488   0.000   0.289   0.289
##      BFI57          0.495   0.076   6.497   0.000   0.495   0.495
##      Conscientiousness =~
##      BFI13          0.032   0.080   0.401   0.689   0.032   0.032
##      BFI18          0.695   0.122   5.690   0.000   0.695   0.695
##      BFI33          0.808   0.122   6.642   0.000   0.808   0.808
##      BFI38          0.187   0.067   2.775   0.006   0.187   0.187
##      BFI53          0.268   0.069   3.868   0.000   0.268   0.268
##      Emotional_Stab =~
##      BFI04          0.583   0.057  10.181   0.000   0.583   0.583
##      BFI09          0.483   0.054   8.916   0.000   0.483   0.483
##      BFI14          0.566   0.054  10.409   0.000   0.566   0.566
##      BFI19          0.588   0.059   9.946   0.000   0.588   0.588
##      BFI24          0.366   0.066   5.550   0.000   0.366   0.366
```

##	BFI29	0.554	0.052	10.711	0.000	0.554	0.554
##	BFI49	0.632	0.051	12.410	0.000	0.632	0.632
##	BFI54	0.711	0.047	15.008	0.000	0.711	0.711
##	Open_Mindedness =~						
##	BFI15	0.367	0.057	6.414	0.000	0.367	0.367
##	BFI20	0.696	0.072	9.653	0.000	0.696	0.696
##	BFI30	0.422	0.074	5.676	0.000	0.422	0.422
##	BFI35	0.688	0.069	9.900	0.000	0.688	0.688
##	BFI40	0.386	0.081	4.741	0.000	0.386	0.386
##	BFI60	0.401	0.062	6.424	0.000	0.401	0.401
##	GFP =~						
##	BFI01	0.548	0.055	9.912	0.000	0.548	0.548
##	BFI06	0.703	0.048	14.775	0.000	0.703	0.703
##	BFI21	0.675	0.047	14.322	0.000	0.675	0.675
##	BFI26	0.460	0.063	7.351	0.000	0.460	0.460
##	BFI36	0.321	0.069	4.669	0.000	0.321	0.321
##	BFI41	0.717	0.039	18.307	0.000	0.717	0.717
##	BFI56	0.574	0.053	10.925	0.000	0.574	0.574
##	BFI02	0.145	0.063	2.303	0.021	0.145	0.145
##	BFI27	0.284	0.064	4.473	0.000	0.284	0.284
##	BFI32	0.660	0.052	12.731	0.000	0.660	0.660
##	BFI57	0.452	0.055	8.288	0.000	0.452	0.452
##	BFI13	0.636	0.056	11.277	0.000	0.636	0.636
##	BFI18	0.204	0.067	3.041	0.002	0.204	0.204
##	BFI33	0.436	0.069	6.354	0.000	0.436	0.436
##	BFI38	0.725	0.042	17.227	0.000	0.725	0.725
##	BFI53	0.690	0.048	14.498	0.000	0.690	0.690
##	BFI04	0.405	0.065	6.204	0.000	0.405	0.405
##	BFI09	0.450	0.059	7.603	0.000	0.450	0.450
##	BFI14	0.268	0.074	3.639	0.000	0.268	0.268
##	BFI19	0.150	0.068	2.224	0.026	0.150	0.150
##	BFI24	0.512	0.066	7.745	0.000	0.512	0.512
##	BFI29	0.479	0.054	8.872	0.000	0.479	0.479
##	BFI49	0.165	0.069	2.403	0.016	0.165	0.165
##	BFI54	0.273	0.069	3.933	0.000	0.273	0.273
##	BFI15	0.660	0.054	12.115	0.000	0.660	0.660
##	BFI20	0.204	0.075	2.709	0.007	0.204	0.204
##	BFI30	0.351	0.068	5.154	0.000	0.351	0.351
##	BFI35	0.397	0.070	5.645	0.000	0.397	0.397
##	BFI40	0.206	0.074	2.778	0.005	0.206	0.206
##	BFI60	0.648	0.051	12.669	0.000	0.648	0.648
##							
##	Covariances:						
##		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
##	Extraversion ~~						
##	Agreeableness	0.000				0.000	0.000
##	Conscientisnss	0.000				0.000	0.000
##	Emotional_Stab	0.000				0.000	0.000
##	Open_Mindednss	0.000				0.000	0.000
##	GFP	0.000				0.000	0.000
##	Agreeableness ~~						
##	Conscientisnss	0.000				0.000	0.000
##	Emotional_Stab	0.000				0.000	0.000
##	Open_Mindednss	0.000				0.000	0.000

```

##      GFP                      0.000                      0.000    0.000
##      Conscientiousness ~~
##      Emotional_Stab          0.000                      0.000    0.000
##      Open_Mindednss          0.000                      0.000    0.000
##      GFP                      0.000                      0.000    0.000
##      Emotional_Stab ~~
##      Open_Mindednss          0.000                      0.000    0.000
##      GFP                      0.000                      0.000    0.000
##      Open_Mindedness ~~
##      GFP                      0.000                      0.000    0.000
##

```

Thresholds:

	Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
## BFI01 t1	-2.602	0.343	-7.589	0.000	-2.602	-2.602
## BFI01 t2	-0.967	0.102	-9.510	0.000	-0.967	-0.967
## BFI01 t3	0.175	0.086	2.036	0.042	0.175	0.175
## BFI06 t1	-2.200	0.225	-9.771	0.000	-2.200	-2.200
## BFI06 t2	-0.482	0.089	-5.407	0.000	-0.482	-0.482
## BFI06 t3	0.631	0.092	6.866	0.000	0.631	0.631
## BFI21 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
## BFI21 t2	-0.222	0.086	-2.578	0.010	-0.222	-0.222
## BFI21 t3	0.719	0.094	7.647	0.000	0.719	0.719
## BFI26 t1	-1.516	0.133	-11.419	0.000	-1.516	-1.516
## BFI26 t2	-0.294	0.087	-3.389	0.001	-0.294	-0.294
## BFI26 t3	0.443	0.089	5.006	0.000	0.443	0.443
## BFI36 t1	-1.383	0.123	-11.249	0.000	-1.383	-1.383
## BFI36 t2	0.210	0.086	2.442	0.015	0.210	0.210
## BFI36 t3	1.150	0.110	10.499	0.000	1.150	1.150
## BFI41 t1	-1.593	0.139	-11.436	0.000	-1.593	-1.593
## BFI41 t2	-0.258	0.087	-2.984	0.003	-0.258	-0.258
## BFI41 t3	0.603	0.091	6.603	0.000	0.603	0.603
## BFI56 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
## BFI56 t2	-0.234	0.086	-2.713	0.007	-0.234	-0.234
## BFI56 t3	0.689	0.093	7.388	0.000	0.689	0.689
## BFI02 t1	-1.847	0.167	-11.090	0.000	-1.847	-1.847
## BFI02 t2	-0.576	0.091	-6.338	0.000	-0.576	-0.576
## BFI02 t3	0.719	0.094	7.647	0.000	0.719	0.719
## BFI27 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
## BFI27 t2	-0.418	0.088	-4.737	0.000	-0.418	-0.418
## BFI27 t3	0.495	0.089	5.541	0.000	0.495	0.495
## BFI32 t1	-2.602	0.343	-7.589	0.000	-2.602	-2.602
## BFI32 t2	-1.085	0.107	-10.187	0.000	-1.085	-1.085
## BFI32 t3	0.070	0.086	0.815	0.415	0.070	0.070
## BFI57 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
## BFI57 t2	-0.058	0.086	-0.679	0.497	-0.058	-0.058
## BFI57 t3	0.828	0.097	8.537	0.000	0.828	0.828
## BFI13 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
## BFI13 t2	-1.106	0.107	-10.293	0.000	-1.106	-1.106
## BFI13 t3	-0.210	0.086	-2.442	0.015	-0.210	-0.210
## BFI18 t1	-1.636	0.143	-11.419	0.000	-1.636	-1.636
## BFI18 t2	-0.282	0.087	-3.254	0.001	-0.282	-0.282
## BFI18 t3	0.674	0.093	7.258	0.000	0.674	0.674
## BFI33 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
## BFI33 t2	-0.456	0.089	-5.140	0.000	-0.456	-0.456

##	BFI33 t3	0.456	0.089	5.140	0.000	0.456	0.456
##	BFI38 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI38 t2	-0.931	0.100	-9.273	0.000	-0.931	-0.931
##	BFI38 t3	0.175	0.086	2.036	0.042	0.175	0.175
##	BFI53 t1	-2.200	0.225	-9.771	0.000	-2.200	-2.200
##	BFI53 t2	-0.812	0.097	-8.412	0.000	-0.812	-0.812
##	BFI53 t3	0.023	0.085	0.272	0.786	0.023	0.023
##	BFI04 t1	-1.085	0.107	-10.187	0.000	-1.085	-1.085
##	BFI04 t2	-0.343	0.087	-3.929	0.000	-0.343	-0.343
##	BFI04 t3	1.325	0.119	11.116	0.000	1.325	1.325
##	BFI09 t1	-1.414	0.125	-11.305	0.000	-1.414	-1.414
##	BFI09 t2	-0.660	0.093	-7.128	0.000	-0.660	-0.660
##	BFI09 t3	0.913	0.100	9.153	0.000	0.913	0.913
##	BFI14 t1	-1.480	0.130	-11.392	0.000	-1.480	-1.480
##	BFI14 t2	-0.646	0.092	-6.997	0.000	-0.646	-0.646
##	BFI14 t3	0.617	0.092	6.734	0.000	0.617	0.617
##	BFI19 t1	-0.967	0.102	-9.510	0.000	-0.967	-0.967
##	BFI19 t2	0.175	0.086	2.036	0.042	0.175	0.175
##	BFI19 t3	1.271	0.116	10.960	0.000	1.271	1.271
##	BFI24 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI24 t2	-1.085	0.107	-10.187	0.000	-1.085	-1.085
##	BFI24 t3	0.431	0.088	4.871	0.000	0.431	0.431
##	BFI29 t1	-1.221	0.113	-10.786	0.000	-1.221	-1.221
##	BFI29 t2	-0.508	0.090	-5.674	0.000	-0.508	-0.508
##	BFI29 t3	0.862	0.098	8.786	0.000	0.862	0.862
##	BFI49 t1	-0.522	0.090	-5.807	0.000	-0.522	-0.522
##	BFI49 t2	0.331	0.087	3.795	0.000	0.331	0.331
##	BFI49 t3	1.414	0.125	11.305	0.000	1.414	1.414
##	BFI54 t1	-1.383	0.123	-11.249	0.000	-1.383	-1.383
##	BFI54 t2	-0.535	0.090	-5.940	0.000	-0.535	-0.535
##	BFI54 t3	0.456	0.089	5.140	0.000	0.456	0.456
##	BFI15 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI15 t2	-0.931	0.100	-9.273	0.000	-0.931	-0.931
##	BFI15 t3	0.222	0.086	2.578	0.010	0.222	0.222
##	BFI20 t1	-1.847	0.167	-11.090	0.000	-1.847	-1.847
##	BFI20 t2	-0.986	0.102	-9.627	0.000	-0.986	-0.986
##	BFI20 t3	-0.140	0.086	-1.629	0.103	-0.140	-0.140
##	BFI30 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI30 t2	-0.734	0.094	-7.776	0.000	-0.734	-0.734
##	BFI30 t3	0.000	0.085	0.000	1.000	0.000	0.000
##	BFI35 t1	-2.355	0.262	-8.984	0.000	-2.355	-2.355
##	BFI35 t2	-1.271	0.116	-10.960	0.000	-1.271	-1.271
##	BFI35 t3	-0.270	0.087	-3.119	0.002	-0.270	-0.270
##	BFI40 t1	-1.682	0.148	-11.381	0.000	-1.682	-1.682
##	BFI40 t2	-0.576	0.091	-6.338	0.000	-0.576	-0.576
##	BFI40 t3	0.294	0.087	3.389	0.001	0.294	0.294
##	BFI60 t1	-1.993	0.187	-10.645	0.000	-1.993	-1.993
##	BFI60 t2	-0.646	0.092	-6.997	0.000	-0.646	-0.646
##	BFI60 t3	0.282	0.087	3.254	0.001	0.282	0.282
##							
##	Variances:						
##		Estimate	Std.Err	z-value	P(> z)	Std.lv	Std.all
##	.BFI01	0.633				0.633	0.633
##	.BFI06	0.382				0.382	0.382

##	.BFI21	0.540	0.540	0.540
##	.BFI26	0.786	0.786	0.786
##	.BFI36	0.878	0.878	0.878
##	.BFI41	0.409	0.409	0.409
##	.BFI56	0.341	0.341	0.341
##	.BFI02	0.331	0.331	0.331
##	.BFI27	0.779	0.779	0.779
##	.BFI32	0.481	0.481	0.481
##	.BFI57	0.551	0.551	0.551
##	.BFI13	0.595	0.595	0.595
##	.BFI18	0.476	0.476	0.476
##	.BFI33	0.158	0.158	0.158
##	.BFI38	0.439	0.439	0.439
##	.BFI53	0.453	0.453	0.453
##	.BFI04	0.496	0.496	0.496
##	.BFI09	0.564	0.564	0.564
##	.BFI14	0.608	0.608	0.608
##	.BFI19	0.632	0.632	0.632
##	.BFI24	0.604	0.604	0.604
##	.BFI29	0.464	0.464	0.464
##	.BFI49	0.574	0.574	0.574
##	.BFI54	0.421	0.421	0.421
##	.BFI15	0.431	0.431	0.431
##	.BFI20	0.474	0.474	0.474
##	.BFI30	0.699	0.699	0.699
##	.BFI35	0.370	0.370	0.370
##	.BFI40	0.809	0.809	0.809
##	.BFI60	0.420	0.420	0.420
##	Extraversion	1.000	1.000	1.000
##	Agreeableness	1.000	1.000	1.000
##	Conscientisnss	1.000	1.000	1.000
##	Emotional_Stab	1.000	1.000	1.000
##	Open_Mindednss	1.000	1.000	1.000
##	GFP	1.000	1.000	1.000

3.2.8 Extração dos escores de GFP Estabilidade Emocional, Abertura e Agradabilidade

```
# 1. Extraíndo escores do modelo Bifactor Orthogonal
# Pegaremos o GFP (Personalidade Geral) e o N_s (Neuroticismo Residual)
# O N_s é importante porque ele ainda manteve confiabilidade (0.627)
bfi_final_scores <- lavPredict(fit_gfp_ortho)

# 2. Adicionando ao banco principal
data_clean$bfi_GFP_Intensity <- bfi_final_scores[, "GFP"]
data_clean$bfi_N_Residual    <- bfi_final_scores[, "Emotional_Stab"]
data_clean$bfi_O_Residual    <- bfi_final_scores[, "Open_Mindedness"]
data_clean$bfi_A_Residual    <- bfi_final_scores[, "Agreeableness"]
```

4 ALMA

4.1 Dados

```
data_clean <- data_clean %>% mutate(subject_id = row_number())

# 1. Criando o formato longo apenas com os números
dados_lavaan_clean <- data_clean %>%
  # Selecionamos o ID e todas as colunas de trecho que NÃO terminam em 'txt'
  select(ID = subject_id, matches("^T\\d{2}_")) %>%
  select(-ends_with("_txt")) %>%
  pivot_longer(
    cols = -ID,
    names_to = c("Trecho", "Item"),
    names_pattern = "T(\\d{2})_(.*)",
    values_to = "Escore"
  ) %>%
  mutate(Escore = as.numeric(Escore)) %>%
  # Voltamos para o formato wide por escuta musical
  pivot_wider(names_from = Item, values_from = Escore) %>%
  # Padronizando os nomes para o seu script de PLS (Iniciais Maiúsculas)
  rename(
    Ativo = ativo, Energizado = energ, Atento = atent,
    Relaxado = relax, Sonolento = sonol, Prazer = prazer,
    Felicidade = felic, Bom_humor = humor, Desagrado = desag,
    Tristeza = trist
  )

# 2. Agora o join com Expertise e as Inversões
# (Assumindo que o objeto 'data' original está carregado)
df_expertise <- data %>%
  select(ID_orig = 1, Expertise_Raw = 9) %>%
  mutate(ID = row_number()) %>% # Garante que o ID bate com o subject_id
  mutate(Expertise = case_when(
    str_detect(Expertise_Raw, "Não estudei") ~ 1,
    str_detect(Expertise_Raw, "De 1 mês a 1 ano") ~ 2,
    str_detect(Expertise_Raw, "De 1 ano a 5 anos") ~ 3,
    str_detect(Expertise_Raw, "5 anos ou mais") ~ 4,
    TRUE ~ 1
  ))

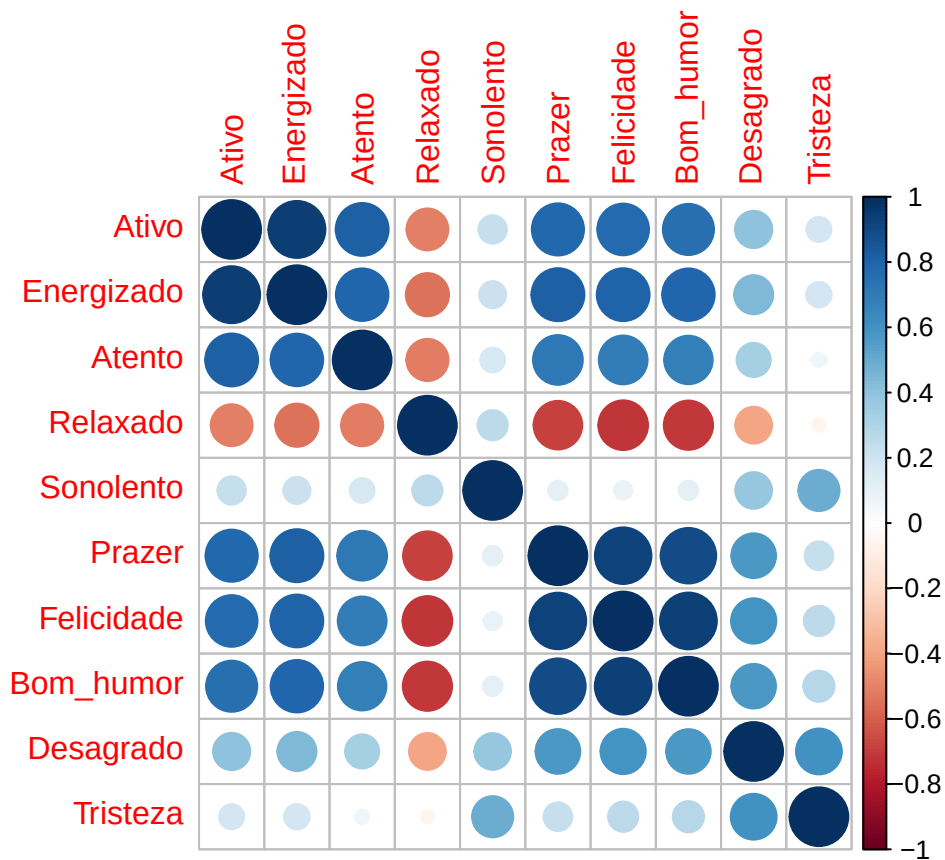
df_hcm <- dados_lavaan_clean %>%
  left_join(df_expertise %>% select(ID, Expertise), by = "ID") %>%
  mutate(
    Relax_Inv = 6 - Relaxado,
    Sono_Inv = 6 - Sonolento,
    Desag_Inv = 6 - Desagrado,
    Trist_Inv = 6 - Tristeza
  ) %>%
  as.data.frame()

library(polycor)
```

```
##
## Anexando pacote: 'polycor'

## O seguinte objeto é mascarado por 'package:psych':
##
##     polyserial

ply_hcnm <- polychoric(df_hcm[3:12])
corrplot(ply_hcnm$rho)
```



4.2 PLS-SEM

```
library(seminr)

## Warning: pacote 'seminr' foi compilado no R versão 4.5.1

##
## Anexando pacote: 'seminr'
```

```
## O seguinte objeto é mascarado por 'package:purrr':  
##  
##     rerun
```

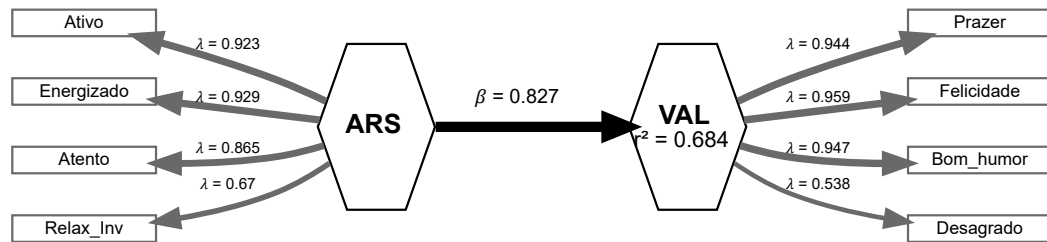
```
# --- ESTÁGIO 1 ---  
mm1 <- constructs(  
  composite("ARS", c("Ativo", "Energizado", "Atento", "Relax_Inv"), weights = mode_A),  
  composite("VAL", c("Prazer", "Felicidade", "Bom_humor", "Desagrado"), weights = mode_A)  
)  
  
pls1 <- estimate_pls(data = df_hcm, measurement_model = mm1,  
  structural_model = relationships(paths(from="ARS", to="VAL")))
```

```
## Generating the seminr model
```

```
## All 2160 observations are valid.
```

```
# --- ESTÁGIO 2 ---  
scores_1a <- as.data.frame(pls1$construct_scores)  
dados_estagio2 <- cbind(scores_1a, Expertise = df_hcm$Expertise)  
  
summ1 <-summary(pls1)  
plot(pls1)
```

```
## file:///C:/Users/vinic/AppData/Local/Temp/RtmpApHhI9/file30c871d249a0/widget30c81d261af1.html screen
```



```
mm2 <- constructs(
  composite("Intensidade", c("ARS", "VAL"), weights = mode_A),
```

```

  composite("Expertise_Ancora", "Expertise", weights = mode_A)
)

pls2 <- estimate_pls(data = dados_estagio2, measurement_model = mm2,
  structural_model = relationships(paths(from="Intensidade", to="Expertise_Ancora")))

## Generating the seminr model

## All 2160 observations are valid.

# RESULTADOS FINAIS
summ <- summary(pls2)

boot_music <- bootstrap_model(pls2, nboot = 1000)

## Bootstrapping model using seminr...

## SEMinR Model successfully bootstrapped

summary(boot_music)

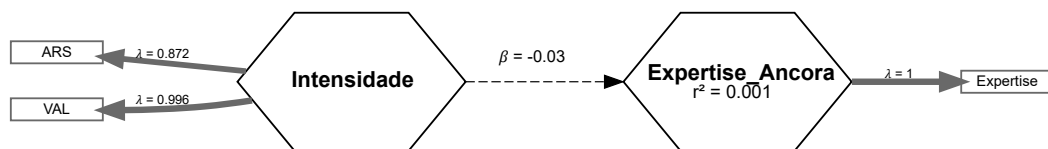
##
## Results from Bootstrap resamples: 1000
##
## Bootstrapped Structural Paths:
## Original Est. Bootstrap Mean Bootstrap SD T Stat. 2.5% CI
## -0.030 -0.026 0.033 -0.919 -0.069
## 97.5% CI
## 0.048
##
## Bootstrapped Weights:
## Original Est. Bootstrap Mean Bootstrap SD
## ARS -> Intensidade 0.154 0.275 0.617
## VAL -> Intensidade 0.869 0.674 0.628
## Expertise -> Expertise_Ancora 1.000 1.000 0.000
## T Stat. 2.5% CI 97.5% CI
## ARS -> Intensidade 0.249 -1.193 1.688
## VAL -> Intensidade 1.384 -1.290 1.711
## Expertise -> Expertise_Ancora . 1.000 1.000
##
## Bootstrapped Loadings:
## Original Est. Bootstrap Mean Bootstrap SD
## ARS -> Intensidade 0.872 0.833 0.197
## VAL -> Intensidade 0.996 0.901 0.208
## Expertise -> Expertise_Ancora 1.000 1.000 0.000
## T Stat. 2.5% CI 97.5% CI
## ARS -> Intensidade 4.437 0.224 0.998
## VAL -> Intensidade 4.785 0.170 1.000
## Expertise -> Expertise_Ancora . 1.000 1.000
##
## Bootstrapped HTMT:

```

```
## Original Est. Bootstrap Mean Bootstrap SD 2.5% CI 97.5% CI
##          0.022          0.028          0.015          0.007          0.065
##
## Bootstrapped Total Paths:
## Original Est. Bootstrap Mean Bootstrap SD 2.5% CI 97.5% CI
##          -0.030          -0.026          0.033          -0.069          0.048
```

```
plot(pls2)
```

```
## file:///C:/Users/vinic/AppData/Local/Temp/RtmpApHhI9/file30c854ec26cb/widget30c818f57957.html screen
```

--- A. Validade Discriminante (Estágio 1) ---
 # O HTMT dirá se ARS e VAL são constructos realmente distintos

```
print(summary(pls1)$validity$htmt)
```

```
##          ARS VAL
## ARS      .      .
## VAL 0.916      .
```

```
# --- B. Multicolinearidade (Estágio 2) ---
# O VIF verificará se há redundância entre os indicadores da Intensidade
# Em modelos de 2ª ordem, queremos VIF < 5 (ideal < 3)
print(summary(pls2)$vif_predecessors)
```

```
## NULL
```

```
# 1. Extraindo o Escore de Intensidade Musical do PLS
# (Cada linha representa uma das 2160 escutas)
score_pls <- as.numeric(pls2$construct_scores[, "Intensidade"])
```

4.3 PCA

```
# 1. Preparando a Matriz de Itens (Usando as inversões que salvaram o PLS)
# Itens: Ativo, Energizado, Atento, Relax_Inv, Prazer, Felicidade, Bom_humor, Desag_Inv, Trist_Inv
itens_triangularacao <- df_hcm %>%
  select(Ativo, Energizado, Atento, Relax_Inv, Prazer, Felicidade, Bom_humor, Desagrado)

# Extraindo o PC1 Não-Rotacionado (A magnitude bruta da variância)
pca_res <- principal(itens_triangularacao, nfactors = 3, rotate = "none")
pca_res$loadings
```

```
##
## Loadings:
##          PC1      PC2      PC3
## Ativo      0.869 -0.290  0.240
## Energizado  0.892 -0.236  0.207
## Atento      0.796 -0.319  0.195
## Relax_Inv   0.676  0.176 -0.607
## Prazer      0.918
## Felicidade  0.920          -0.149
## Bom_humor   0.908  0.105 -0.127
## Desagrado   0.471  0.761  0.421
##
##          PC1      PC2      PC3
## SS loadings  5.377  0.876  0.731
## Proportion Var 0.672  0.109  0.091
## Cumulative Var 0.672  0.782  0.873
```

```
score_pca <- as.numeric(pca_res$scores[, 1])
```

4.4 Análise Exploratória de Grafos Hierárquica

```
library(EGAnet)

hega_res <- hierEGA(itens_triangularacao, algorithm = "louvain")
score_hega <- as.numeric(rowMeans(hega_res$parameters$lower_scores))
```

4.5 CFA

```
itens_finais <- df_hcm %>%
  select(Ativo, Energizado, Atento, Relax_Inv, Prazer, Felicidade, Bom_humor, Desagrado)
```

```
mod_cfa_bifactor <- '
  Intensity =~ Ativo + Energizado + Atento + Relax_Inv + Prazer +
  Felicidade + Bom_humor + Desagrado

  Arousal =~ Ativo + Energizado + Atento + Relax_Inv
  Valence =~ Prazer + Felicidade + Bom_humor + Desagrado
'

fit_cfa_final <- cfa(mod_cfa_bifactor, data = df_hcm, orthogonal = TRUE,
  ordered = TRUE, estimator = "WLSMV")

fitMeasures(fit_cfa_final, fit.measures = c("chisq", "df", "pvalue", "cfi", "rmsea",
  "rmsea.ci.lower", "rmsea.ci.upper", "srmr"))
```

##	chisq	df	pvalue	cfi	rmsea
##	19.051	12.000	0.087	1.000	0.016
##	rmsea.ci.lower	rmsea.ci.upper	srmr		
##	0.000	0.030	0.011		

```
#modindices(fit_cfa_final, sort. = T)
compRelSEM(fit_cfa_final)
```

##	Intensity	Arousal	Valence
##	0.901	0.039	0.151

```
score_cfa <- lavPredict(fit_cfa_final) # Escore Fatorial Latente
score_cfa <- as.data.frame(score_cfa)
```

4.5.1 Falha dos modelos não bifactor

```
mod_cfa <- '

  Arousal =~ Ativo + Energizado + Atento + Relax_Inv
  Valence =~ Prazer + Felicidade + Bom_humor + Desagrado
```

```
'
fit_cfa <- cfa(mod_cfa, data = df_hcm, orthogonal = FALSE,
              ordered = TRUE, estimator = "WLSMV")

fitMeasures(fit_cfa, fit.measures = c("chisq","df","pvalue","cfi", "rmsea",
                                     "rmsea.ci.lower", "rmsea.ci.upper","srmr"))
```

```
##          chisq          df          pvalue          cfi          rmsea
##      526.695        19.000          0.000          0.999          0.111
## rmsea.ci.lower rmsea.ci.upper          srmr
##          0.103          0.120          0.059
```

```
#modindices(fit_cfa_final, sort. = T)
compRelSEM(fit_cfa)
```

```
## Arousal Valence
##    0.976    0.912
```

```
mod_cfa <- '
  FG =~ Arousal + Valence

  Arousal =~ Ativo + Energizado + Atento + Relax_Inv
  Valence =~ Prazer + Felicidade + Bom_humor + Desagrado
```

```
'
fit_cfa <- cfa(mod_cfa, data = df_hcm, orthogonal = FALSE,
              ordered = TRUE, estimator = "WLSMV")
```

```
## Warning: lavaan->lav_model_vcov():
##    Could not compute standard errors! The information matrix could not be
##    inverted. This may be a symptom that the model is not identified.
```

```
## Warning: lavaan->lav_test_satorra_bentler():
##    could not invert information matrix needed for robust test statistic
```

```
fitMeasures(fit_cfa, fit.measures = c("chisq","df","pvalue","cfi", "rmsea",
                                     "rmsea.ci.lower", "rmsea.ci.upper","srmr"))
```

```
##          chisq          df          pvalue          cfi          rmsea
##      526.695        18.000          0.000          0.999          0.114
## rmsea.ci.lower rmsea.ci.upper          srmr
##          0.106          0.123          0.059
```

```
#modindices(fit_cfa_final, sort. = T)
compRelSEM(fit_cfa, higher = "FG")
```

```
## Arousal Valence      FG
##    0.976    0.912    0.899
```

4.5.2 Validade da soma dos escores

```
# 1. Calcular o escore somado bruto (Adição simples dos 8 itens)
df_hcm$Soma_Bruta <- df_hcm$Ativo + df_hcm$Energizado + df_hcm$Atento +
  df_hcm$Relax_Inv + df_hcm$Prazer + df_hcm$Felicidade +
  df_hcm$Bom_humor + df_hcm$Desagrado

# 2. Criar um dataframe juntando a Soma Bruta com as extrações latentes
df_triangulacao_total <- data.frame(
  PCA = score_pca,
  hEGA = score_hega,
  PLS = score_pls,
  CFA = score_cfa$Intensity
)

df_validade_pratica <- data.frame(
  Soma_Bruta = df_hcm$Soma_Bruta,
  CFA_Intensidade = score_cfa$Intensity,
  PLS_Intensidade = df_triangulacao_total$PLS
)

# 3. Calcular a Matriz de Correlação (Spearman, por conta da soma ordinal)
matriz_soma <- cor(df_validade_pratica, method = "spearman")

cat("\n>>> CORRELAÇÃO: SOMA BRUTA vs. ESCORES COMPLEXOS <<<\n")

##
## >>> CORRELAÇÃO: SOMA BRUTA vs. ESCORES COMPLEXOS <<<

print(round(matriz_soma, 3))

##
##          Soma_Bruta CFA_Intensidade PLS_Intensidade
## Soma_Bruta      1.000          0.985          0.980
## CFA_Intensidade  0.985          1.000          0.951
## PLS_Intensidade  0.980          0.951          1.000

MVN::mvn(df_triangulacao_total, univariate_test = "SW")$univariate_normality

##          Test Variable Statistic p.value   Normality
## 1 Shapiro-Wilk      PCA      0.971 <0.001 Not normal
## 2 Shapiro-Wilk     hEGA      0.964 <0.001 Not normal
## 3 Shapiro-Wilk      PLS      0.961 <0.001 Not normal
## 4 Shapiro-Wilk      CFA      0.982 <0.001 Not normal

# 3. Matriz de Correlação Spearman
matriz_finalissima <- cor(df_triangulacao_total, method = "spearman", use = "pairwise.complete.obs")
print(round(matriz_finalissima, 3))

##          PCA hEGA  PLS  CFA
## PCA  1.000 0.999 0.979 0.987
## hEGA 0.999 1.000 0.980 0.984
## PLS  0.979 0.980 1.000 0.951
## CFA  0.987 0.984 0.951 1.000
```

4.6 Relações entre os modelos de mensuração

```
library(tidyverse)
library(lme4)

## Carregando pacotes exigidos: Matrix

##
## Anexando pacote: 'Matrix'

## Os seguintes objetos são mascarados por 'package:tidyr':
##
##     expand, pack, unpack

library(lmerTest)

##
## Anexando pacote: 'lmerTest'

## O seguinte objeto é mascarado por 'package:lme4':
##
##     lmer

## O seguinte objeto é mascarado por 'package:stats':
##
##     step

# 1. Reconstruindo o banco longo com TODAS as variáveis necessárias
# Repetimos cada linha de data_clean 10 vezes para alinhar com os 10 trechos musicais
data_long_completo <- data_clean %>%
  select(subject_id, age, sex, education_grouped,
         bfi_GFP_Intensity, bfi_N_Residual, bfi_O_Residual, bfi_A_Residual, dass_p_factor) %>%
  slice(rep(1:n(), each = 10)) %>%
  mutate(excerpt_id = rep(paste0("T", sprintf("%02d", 1:10)), times = 216))

# 2. Anexando os escores padronizados da triangulação (2160 linhas)
# (Usamos o df_triangulacao_total que você já validou)
data_long_completo$score_PCA <- as.numeric(scale(df_triangulacao_total$PCA))
data_long_completo$score_hEGA <- as.numeric(scale(df_triangulacao_total$hEGA))
data_long_completo$score_PLS <- as.numeric(scale(df_triangulacao_total$PLS))
data_long_completo$score_CFA <- as.numeric(scale(df_triangulacao_total$CFA))

# Definindo a fórmula definitiva
formula_full <- " ~ bfi_GFP_Intensity + bfi_N_Residual + bfi_O_Residual + bfi_A_Residual +
               dass_p_factor + age + education_grouped + sex +
               (1 | subject_id) + (1 | excerpt_id)"

# Rodando os modelos
fit_pca <- lmer(as.formula(paste0("score_PCA", formula_full)), data = data_long_completo)
fit_hega <- lmer(as.formula(paste0("score_hEGA", formula_full)), data = data_long_completo)
```

```
fit_pls <- lmer(as.formula(paste0("score_PLS", formula_full)), data = data_long_completo)
fit_cfa <- lmer(as.formula(paste0("score_CFA", formula_full)), data = data_long_completo)
```

```
# 3. Comparação de Performance com o padrão que você gosta
library(performance)
```

```
## Warning: pacote 'performance' foi compilado no R versão 4.5.2
```

```
compare_performance(fit_pca, fit_hega, fit_cfa, fit_pls, metrics = "common")
```

```
## When comparing models, please note that probably not all models were fit
## from same data.
```

```
## # Comparison of Model Performance Indices
```

```
##
## Name      |           Model | AIC (weights) | BIC (weights) | R2 (cond.)
## -----
## fit_pca   | lmerModLmerTest | 5163.4 (<.001) | 5248.6 (<.001) |      0.497
## fit_hega  | lmerModLmerTest | 5167.4 (<.001) | 5252.5 (<.001) |      0.496
## fit_cfa   | lmerModLmerTest | 5090.5 (>.999) | 5175.6 (>.999) |      0.521
## fit_pls   | lmerModLmerTest | 5216.2 (<.001) | 5301.3 (<.001) |      0.475
##
```

```
## Name      | R2 (marg.) | ICC | RMSE
## -----
## fit_pca   |      0.077 | 0.455 | 0.689
## fit_hega  |      0.078 | 0.453 | 0.690
## fit_cfa   |      0.080 | 0.479 | 0.672
## fit_pls   |      0.076 | 0.432 | 0.705
```

```
#library(sjPlot)
```

```
# Criando uma tabela única comparando os Betas dos 5 modelos
#tab3 <- tab_model(fit_pca, fit_hega, fit_cfa, fit_pls,
#                   show.ci = FALSE, show.se = TRUE,
#                   dv.labels = c("PCA", "hEGA", "CFA", "PLS"),
#                   title = "Estabilidade dos Preditores em Diferentes Métodos de #Extração")
```

```
#tab3
```

```
sessionInfo()
```

```
## R version 4.5.0 (2025-04-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Portuguese_Brazil.utf8 LC_CTYPE=Portuguese_Brazil.utf8
```

```

## [3] LC_MONETARY=Portuguese_Brazil.utf8 LC_NUMERIC=C
## [5] LC_TIME=Portuguese_Brazil.utf8
##
## time zone: America/Sao_Paulo
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] performance_0.15.2 lmerTest_3.1-3      lme4_1.1-37         Matrix_1.7-3
## [5] EGAnet_2.3.0       semnr_2.3.7         polycor_0.8-1       eRm_1.0-10
## [9] semTools_0.5-7     semPlot_1.1.7       lavaan_0.6-20       corrplot_0.95
## [13] lsr_0.5.2          psych_2.5.3         readxl_1.4.5        lubridate_1.9.4
## [17] forcats_1.0.1      stringr_1.6.0       dplyr_1.1.4         purrr_1.2.1
## [21] readr_2.1.6        tidyr_1.3.1         tibble_3.3.1        ggplot2_4.0.2
## [25] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] splines_4.5.0      later_1.4.2         cellranger_1.1.0
## [4] XML_3.99-0.20      rpart_4.1.24        lifecycle_1.0.5
## [7] Rdpack_2.6.4       processx_3.8.6      lattice_0.22-7
## [10] MASS_7.3-65        insight_1.4.2       rockchalk_1.8.157
## [13] backports_1.5.0    magrittr_2.0.4      openxlsx_4.2.8.1
## [16] Hmisc_5.2-5        plotly_4.11.0       rmarkdown_2.30
## [19] yaml_2.3.10        qgraph_1.9.8        zip_2.3.3
## [22] pbapply_1.7-4      chromote_0.5.1       minqa_1.2.8
## [25] RColorBrewer_1.1-3 multcomp_1.4-29      abind_1.4-8
## [28] quadprog_1.5-8     nnet_7.3-20         TH.data_1.1-5
## [31] webshot2_0.1.2     sandwich_3.1-1      moments_0.14.1
## [34] nortest_1.0-4      arm_1.14-4          codetools_0.2-20
## [37] energy_1.7-12      tidysselect_1.2.1   shape_1.4.6.1
## [40] farver_2.1.2       viridis_0.6.5       stats4_4.5.0
## [43] base64enc_0.1-3    webshot_0.5.5       jsonlite_2.0.0
## [46] mitml_0.4-5        Formula_1.2-5       survival_3.8-3
## [49] iterators_1.0.14   emmeans_2.0.0       foreach_1.5.2
## [52] tools_4.5.0        Rcpp_1.1.0          glue_1.8.0
## [55] mnormt_2.1.1       gridExtra_2.3       pan_1.9
## [58] xfun_0.52          admisc_0.39         websocket_1.4.4
## [61] numDeriv_2016.8-1.1 withr_3.0.2         fastmap_1.2.0
## [64] MVN_6.2            boot_1.3-32         digest_0.6.38
## [67] mi_1.2             timechange_0.3.0    R6_2.6.1
## [70] estimability_1.5.1 mice_3.17.0         colorspace_2.1-1
## [73] gtools_3.9.5       jpeg_0.1-11         Diagrammer_1.0.11
## [76] utf8_1.2.6         generics_0.1.4      data.table_1.17.0
## [79] corpcor_1.6.10     httr_1.4.7          htmlwidgets_1.6.4
## [82] pkgconfig_2.0.3    sem_3.1-16          gtable_0.3.6
## [85] S7_0.2.1           htmltools_0.5.8.1   carData_3.0-5
## [88] scales_1.4.0       png_0.1-8           reformulas_0.4.2
## [91] knitr_1.50         rstudioapi_0.17.1   tzdb_0.5.0
## [94] reshape2_1.4.5     coda_0.19-4.1       visNetwork_2.1.4
## [97] checkmate_2.3.2    nlme_3.1-168        nloptr_2.2.1
## [100] zoo_1.8-14         parallel_4.5.0      foreign_0.8-90
## [103] pillar_1.11.1      grid_4.5.0          vctrs_0.6.5

```


## [106] promises_1.3.2	car_3.1-3	OpenMx_2.22.10
## [109] jomo_2.7-6	xtable_1.8-4	cluster_2.1.8.1
## [112] htmlTable_2.4.3	evaluate_1.0.5	pbivnorm_0.6.0
## [115] mvtnorm_1.3-3	cli_3.6.5	kutils_1.73
## [118] compiler_4.5.0	rlang_1.1.6	labeling_0.4.3
## [121] fdrtool_1.2.18	ps_1.9.1	plyr_1.8.9
## [124] stringi_1.8.7	viridisLite_0.4.3	gsl_2.1-9
## [127] lazyeval_0.2.2	lisrelToR_0.3	glmnet_4.1-8
## [130] hms_1.1.4	glasso_1.11	rbibutils_2.3
## [133] igraph_2.2.1	broom_1.0.10	RcppParallel_5.1.11-1